UniTs - University of Trieste

Faculty of Scientific and Data Intensive Computing

Department of mathematics informatics and geosciences

Advanced High Performance Computing

Lecturer:
Prof. Luca Tornatore

Author:
Andrea Spinelli

April 22, 2026

github.com/Spina02 andreaspinelli2002@gmail.com

Preface

In this course, we will explore a set of fundamental topics essential for high-performance and parallel computing:

- **Vectorization**: methods for processing multiple data elements at once to fully utilize modern CPU architectures.
- **OpenMP**: a popular interface for developing shared-memory parallel programs that allows straightforward multi-threading.
- **GPU**:
  - **cuda**: NVIDIA's platform and API for general-purpose computing on GPUs.
  - **openacc**: a directive-based standard for simplifying the acceleration of code on GPUs and other devices.
- **MPI**:
  - **Data-Types**: constructing and efficiently communicating custom and built-in datatypes in distributed environments.
  - **I/O**: strategies and tools for performing parallel input and output in large-scale applications.
  - **one-sided communication**: advanced features for direct remote memory access and asynchronous messaging.
- **MPI in python**: using the Message Passing Interface within the Python ecosystem for approachable and flexible parallel programming.

Contents

1 Vectorization

1.1 Basic Concepts

Modern high-performance computing systems exploit multiple layers of parallelism to achieve maximum performance. At the largest scale, clusters or supercomputers are made up of many nodes working together. Within each node, there may be several CPUs (sockets), each containing multiple processing cores capable of running tasks concurrently.

A further dimension in modern HPC is ***offloading***, where compute-intensive tasks and associated data are delegated from the CPU to accelerators such as GPUs (Graphics Processing Units). Offloading entails transferring data across the system's architecture, a process whose performance impact depends on the system's memory configuration and interconnect.

- In **NVIDIA-based systems**, CPUs typically access DDR5 RAM, while GPUs benefit from their own high-bandwidth HBM3 memory. Data movement across the CPU-GPU boundary (usually via PCIe or NVLink) can be a bottleneck and must be carefully optimized.
- In contrast, **AMD's recent architectures** (notably with some Instinct GPUs and EPYC CPUs) may feature a unified DDR5 memory pool shared by both CPU and GPU, reducing data transfer overheads and enabling more direct sharing.

In addition to thread and process-level parallelism, modern processors employ a crucial form of parallelism: **vectorization**. Vectorization embodies ***Single Instruction Multiple Data (SIMD)*** execution, a paradigm in which a single instruction operates on multiple data elements simultaneously using specialized hardware known as ***vector registers***.

Vector registers (VREGs) are designed for operations on aggregated data, they are considerably wider than traditional scalar registers, commonly supporting 128, 256, or even 512 bits at a time.

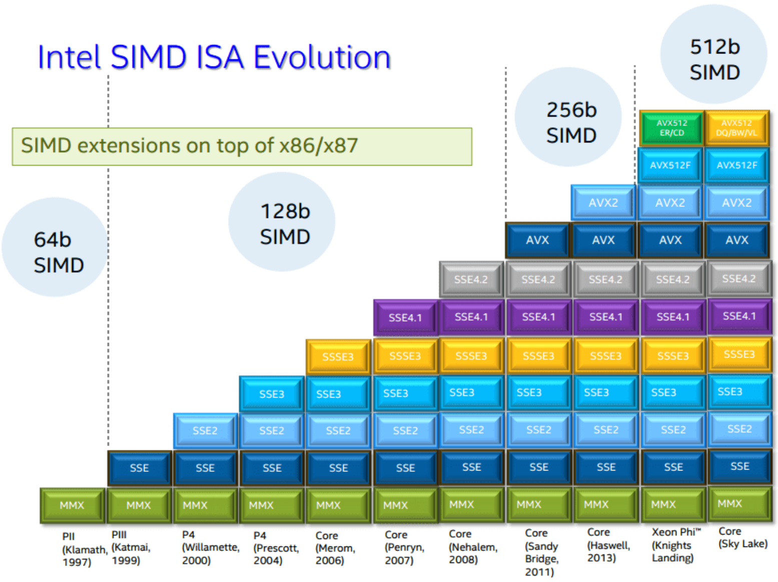


Figure 1.1: Intel SIMD ISA Evolution: timeline of vector instruction set extensions introduced with each CPU generation. The diagram shows how SIMD register width increased from MMX and SSE (64-128 bits) to AVX (256 bits) and AVX-512 (512 bits), with each new standard supporting more data parallelism.

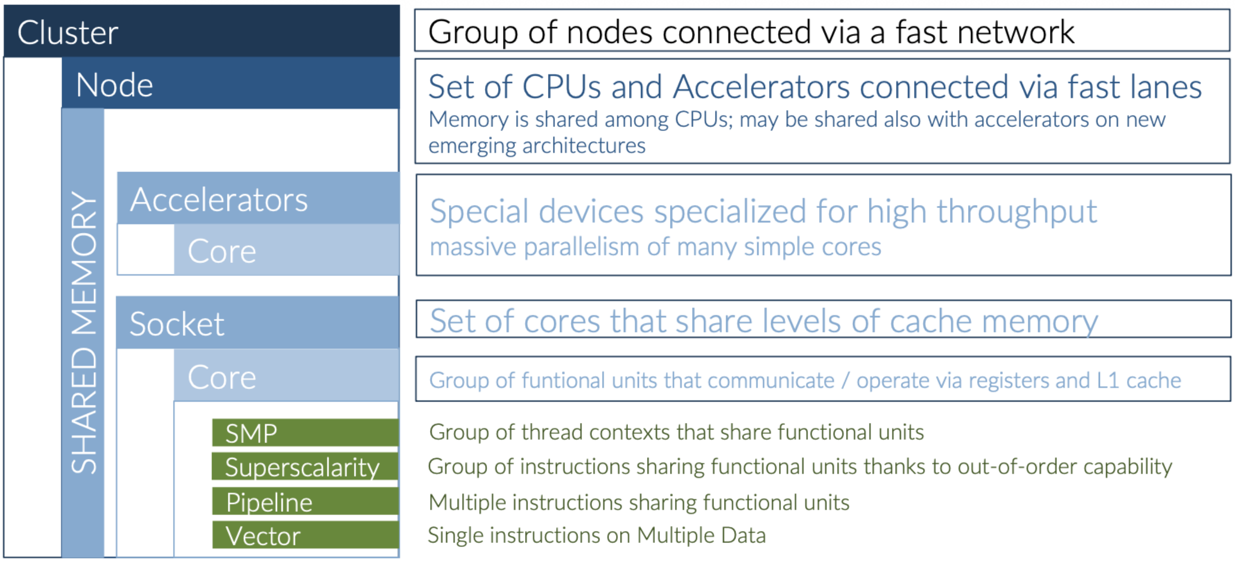


Figure 1.2: The architecture of a modern HPC cluster.

Modern x86 CPUs have evolved both in structure and capacity. The main idea is straightforward: the wider the register, the more data elements you can work on at once with a single instruction. Intel has introduced three generations of SIMD registers:

- **SSE (128-bit):** Each register (`xmm0`) can hold either 2 double-precision (64-bit) values or 4 single-precision (32-bit) floating-point values. This was the first major step for SIMD on x86.
- **AVX (256-bit):** The register width doubles. Now, a `ymm0` register can operate on 4 doubles or 8 floats at once, effectively doubling the amount of data processed per instruction.
- **AVX-512 (512-bit):** The register width doubles yet again. The `zmm0` register can simultaneously process up to 8 doubles or 16 floats.

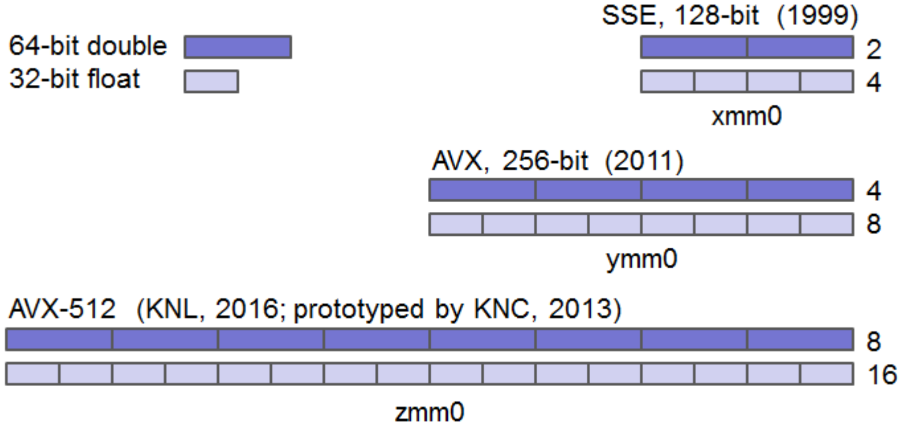


Figure 1.3: SIMD registers: each generation widens the registers, allowing more doubles (dark) or floats (light) to be processed in parallel. [**cornellCornellVirtual**]

Wider registers over the generations (SSE $\rightarrow$ AVX $\rightarrow$ AVX-512) translate directly into the ability to operate on more data elements at once. This increase in register width is key to the dramatic gains in performance for vectorizable workloads, as it enables each instruction to do more useful work in parallel.

How to achieve vectorization?

Vectorization can be approached at various levels, from the most convenient and portable to the most hardware-specific and optimized:

- **Explicit compiler pragmas** (e.g., `#pragma omp simd`, `#pragma vector`): Code annotations that help the compiler vectorize loops or code regions. These are high-level, portable, require few changes, but give only moderate control.
- **Auto-vectorization by the compiler**: Compilers can often detect and apply SIMD to loops with the right options (`-O2`, `-ftree-vectorize`, etc). Convenient, needs no source changes, but gives little control and can miss opportunities.
- **C++ vector libraries**: Libraries such as `std::valarray`, `Vc`, or `Eigen` let you write high-level vector code, balancing portability and some SIMD control.
- **Compiler-specific vector extensions**: Using types like `__m256d` or `float32x4_t` gives direct SIMD register access, improving performance but reducing portability.
- **Intrinsics**: Functions mapping to hardware SIMD instructions (e.g., `_mm256_add_pd`), allow very fine-grained, fast vector code at the cost of readability and portability.
- **Handwritten assembly**: Writing assembly gives maximum control and performance, but is not portable and is complex and error-prone.

Higher-level approaches are simpler and portable; lower-level ones give more control and tuning, but cost portability and add complexity.

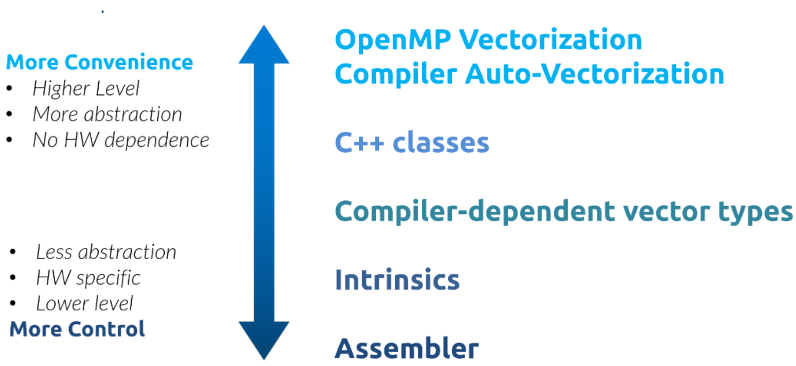


Figure 1.4: Approaches to vectorize, from high-level convenience to low-level control. [**ADV_HPC**]

Different architectures

Vector ISAs follow two main philosophies. The traditional x86 ISA uses **fixed-width** vectors, while ARM and RISC-V adopt a **scalable** design where the vector length is not fixed in the binary.

Feature	x86 (AVX/AVX-512)	ARM (SVE)	RISC-V RVV
Philosophy	***Fixed-width*** (128/256/512 bit)	***Scalable*** (128–2048 bit)	***Scalable*** (undefined VLEN)
Binary model	***Fragmented*** (compile for each target)	***VLA*** (Vector-Length-Agnostic) write once, run anywhere	***VLA*** write once, run anywhere
Register count	16 (AVX2) 32 (AVX-512)	32	32 + LMUL grouping
Registers flexibility	None (fixed per register/size)	None (set at HW level, 128–2048 bit)	LMUL (register grouping)
Predication	8 mask registers (AVX-512 add-on)	16 predicate registers (core feature)	Built-in mask operand (often `v0`)
Configuration	***Implicit*** (by the called instruction)	***Implicit*** (type chosen; length is loop-invariant)	***Explicit*** via `vsetvl`
ISA complexity	***High fragmentation*** (AVX-512F, CD, VL, ...)	***Low fragmentation*** (NEON, SVE, SVE2)	***Unified*** (RVV 1.0)
Hardware cost	Can cause clock throttling (AVX-512)	Designed for power/performance scaling	—

- **Fixed vs Scalable size**: On x86_64 you must target a specific register width (SSE 128, AVX 256, AVX-512 512) and call the corresponding instructions or binaries. This forces specialized code paths and ***fragmentation***. With scalable architectures (SVE, RVV) you write ***Vector-Length-Agnostic (VLA)*** code: the same loop adapts to the hardware at run time, running efficiently on a 128-bit core or on a server with 1024-bit registers, greatly reducing fragmentation.
- **Native vs add-on predication**: Predication lets a vector instruction conditionally process elements without branching, by creating a boolean *mask* to decide which lanes are active.

```
for (int i = 0; i < N; ++i)
    if (data[i] > 0.0)
        data[i] *= 2.0;
```

  If the mask for an 8-lane vector is, for instance:

```
mask: [ 0 0 1 1 1 0 1 1 ]
data: [ u u p p p u p p ]  // u = unchanged, p = processed
```

  The inactive lanes (0) are left untouched, while the active lanes (1) are updated. In practice there are two ways to realize this:
  – skip the false lanes so they are not executed at all; or
  – execute all lanes but ignore the results of the false lanes ("blend" them with the old values).
  SVE provides **native predication** via 16 predicate registers and rich lane-wise logic; RVV uses a built-in mask operand (commonly `v0`) as part of the ISA. In x86, predication is an *add-on* of AVX-512 (8 mask registers), while up to AVX2 it is emulated via blending operations.

1.2 Checking for SIMD capabilities

You can check **statically** the value for your cpu, for instance by grepping the output of either `lscpu` or `/proc/cpuinfo`. But what if you want to check the capabilities of the architecture where the code is running? We can use multiple methods to check the capabilities of the architecture:

- **Compiler built-in functions**: The compiler provides built-in functions to check the capabilities of the architecture.

```c
#include <cpuid.h>

int main() {
    __builtin_cpu_init();

    if (__builtin_cpu_supports("avx512f"))
        ...
    if (__builtin_cpu_supports("avx2"))
        ...
    if (__builtin_cpu_supports("avx"))
        ...
    if (__builtin_cpu_supports("sse4.2"))
        ...
    if (__builtin_cpu_supports("sse4.1"))
        ...
    if (__builtin_cpu_supports("sse3"))
        ...
```

- **Compiler intrinsics**: The compiler provides intrinsics to check the capabilities of the architecture.

```c
#include <immintrin.h>

#ifdef __AVX512__
#define V_DSIZE (sizeof( __m512d ) / sizeof(double) )

#elif defined ( __AVX__ ) || defined ( __AVX2__ )
#define V_DSIZE (sizeof( __m256d ) / sizeof(double) )

#elif defined ( __SSE4__ ) || defined ( __SSE3__ )
#define V_DSIZE (sizeof( __m128d ) / sizeof(double) )

#else
#define V_DSIZE 1UL
#endif
```

Tip: `-march=native`

By default, the compiler is able to correctly recognize the CPU's capabilities but actually gets a wrong vector size. We need to use the `-march=native` flag, since it will not be able to detect the vector capabilities of the architecture.

In some cases, you might need to explicitly target a particular SIMD extension (for either Intel or AMD architectures). This can be accomplished by specifying the appropriate compiler flags:

- `-mavx512f` : Enables AVX-512 instructions.
- `-mavx2` : Enables AVX2 instructions.
- `-mavx` : Enables AVX instructions.
- `-msse4.2` : Enables SSE4.2 instructions.
- `-msse4.1` : Enables SSE4.1 instructions.
- `-msse3` : Enables SSE3 instructions.

1.3 Vector Types via Intrinsics

Once we include the `<immintrin.h>` header, we get access to the intrinsincs routines and types. **Intrinsics** are functions that are provided by the compiler to allow the programmer to directly use the vector instructions of the architecture.

INTEGER types

types name	int 8bits	int 16bits	int 32bits	int 64bits
`__m64`	8x	4x	2x	1x
`__m128i`	16x	8x	4x	2x
`__m256i`	32x	16x	8x	4x
`__m512i`	64x	32x	16x	8x

FLOATING-POINT types

types name	single precision	double precision
`__m128`	4x	–
`__m128d`	–	2x
`__m256`	8x	–
`__m256d`	–	4x
`__m512`	16x	–
`__m512d`	–	8x

To make explicit use of vector types while keeping the code portable across machines, it is convenient to introduce a small layer of typedefs and wrappers that adapts at compile time to the ISA detected by the compiler. The names `dvector_t` , `fvector_t` and `ivector_t` will always exist in our code with the correct width, and a couple of macros expose their element count and bit size.

```
// Select vector types based on the available ISA
#ifdef __AVX512__
typedef __m512d dvector_t;
typedef __m512  fvector_t;
typedef __m512i ivector_t;

#elif defined ( __AVX__ ) || defined ( __AVX2__ )
typedef __m256d dvector_t;
typedef __m256  fvector_t;
typedef __m256i ivector_t;

#elif defined ( __SSE4__ ) || defined ( __SSE3__ )
typedef __m128d dvector_t;
typedef __m128  fvector_t;
typedef __m128i ivector_t;

#else
typedef double dvector_t;
typedef float  fvector_t;
typedef int    ivector_t;
#endif

// Convenience macros for sizes (elements per vector and bit width)
#define DV_ELEMENT_SIZE ( sizeof(dvector_t) / sizeof(double) )
#define DV_BIT_SIZE     ( sizeof(dvector_t) * 8 )
#define FV_ELEMENT_SIZE ( sizeof(fvector_t) / sizeof(float) )
#define FV_BIT_SIZE     ( sizeof(fvector_t) * 8 )
#define IV_ELEMENT_SIZE ( sizeof(ivector_t) / sizeof(int) )
#define IV_BIT_SIZE     ( sizeof(ivector_t) * 8 )
```

In real codes one typically hides the operations behind tiny wrappers so that call sites stay uniform: a short `#ifdef` region selects the proper `SSE` / `AVX` / `AVX-512` intrinsic. As an example, below we provide a simple "vector summation" for doubles, floats and integers, via `VDSUM` , `VFSUM` and `VISUM` . We will return to intrinsics usage later in the chapter.

Example: ***Vector summation***

Wrappers for a "vector summation" on doubles, floats and ints:

```c
#ifdef __AVX512__
#define VDSUM(v1, v2, r) ( (r) = _mm512_add_pd( (v1), (v2) ) )
#define VFSUM(v1, v2, r) ( (r) = _mm512_add_ps( (v1), (v2) ) )
#define VISUM(v1, v2, r) ( (r) = _mm512_add_epi32( (v1), (v2) ) )
#elif defined ( __AVX__ ) || defined ( __AVX2__ )
#define VDSUM(v1, v2, r) ( (r) = _mm256_add_pd( (v1), (v2) ) )
#define VFSUM(v1, v2, r) ( (r) = _mm256_add_ps( (v1), (v2) ) )
#define VISUM(v1, v2, r) ( (r) = _mm256_add_epi32( (v1), (v2) ) )
#elif defined ( __SSE4__ ) || defined ( __SSE3__ )
#define VDSUM(v1, v2, r) ( (r) = _mm_add_pd( (v1), (v2) ) )
#define VFSUM(v1, v2, r) ( (r) = _mm_add_ps( (v1), (v2) ) )
#define VISUM(v1, v2, r) ( (r) = _mm_add_epi32( (v1), (v2) ) )
#else
#define VDSUM(v1, v2, r) ( (r) = (v1) + (v2) )
#define VFSUM(v1, v2, r) ( (r) = (v1) + (v2) )
#define VISUM(v1, v2, r) ( (r) = (v1) + (v2) )
#endif
```

Warning: ***Production-ready headers***

The approach we have just discussed is intended for didactical purposes. For comprehensive, production-ready headers, consider for instance:

- Agner Fog's VectorClass Library (VCL)
- Google's Highway

1.4 Loops auto-vectorization

Auto-vectorization refers to the compiler's ability to automatically convert suitable loops and code blocks into vector instructions, allowing data-level parallelism and boosting performance even without manual use of SIMD intrinsics. While this is a powerful tool, it is important to be aware that successful auto-vectorization is not always guaranteed. Various elements can limit or block this process, such as:

- loop-carried data dependencies
- complex control dependencies or conditional branches within the loop
- unaligned or poorly aligned memory accesses
- loop structures that do not match vectorization patterns
- strided or irregular memory access patterns
- calls to functions that cannot be inlined or vectorized
- use of math functions that lack vectorizable implementations
- data types unsupported by the target SIMD ISA
- …

It's important to keep these obstacles in mind when writing code that we want to auto-vectorize.

To ask the compiler to vectorize loops computations, we can use the following flags:

Compiler	Flag	Description
GCC	`-ftree-vectorize`	enables loops vectorization
	`-funroll-loops`	enables the loop unrolling (may or may not issue faster code)
	`-march=native`	specifies the current one as target architecture
	`-mtune=native`	ask maximum code tuning for the host architecture
clang	`-mllvm`	(uses LLVM's internal vectorization)
	`-force-vector-width=4`	enforces vector width (e.g., 4) for SIMD instructions
Intel	`-xHost`	enables maximum vectorization available on the target cpu
	`-axFLAG1 -axFLAG2 ...`	enables code for multiple targets

If we want to ask the compiler to report on optimizations and vectorizations, we can use the following flags:

Compiler	Flag	Description
GCC	`-fopt-info-vec-optimized`	reports on the successful vectorizations
	`-fopt-info-vec-missed`	reports on the reasons for unsuccessful vectorization
	`-fopt-info-vec-all`	reports all vectorization optimizations
clang	`-Rpass=loop-vectorize`	identifies loops that were successfully vectorized
	`-Rpass-missed=loop-vectorize`	identifies loops that were NOT successfully vectorized
	`-Rpass-analysis=loop-vectorize`	identifies statements that caused vectorization to fail (with `-fsave-optimization-record`, detailed causes may be listed)
Intel	`-qopt-report=n`	enables diagnostic output; n=1: vectorized, n=2: non-vectorized+reasons, n=3: dependency info, n=4: only vectorization, n=5: dependency issues
	`-qopt-report-file=`*`name`*	writes the report to the specified file

In addition to enabling vectorization flags, we can provide the compiler with additional directives and hints to facilitate and optimize code vectorization:

Compiler	Pragma	Description
GCC	`#pragma GCC ivdep`	Ignore potential loop-carried dependencies that are not formally proven
	`#pragma GCC unroll n`	Unroll the subsequent loop body `n` times
clang	`#pragma clang ivdep`	Ignore potential loop-carried dependencies
	`#pragma clang vectorize(enable|disable)`	Enable/disable auto-vectorization of the loop
	`#pragma clang vectorize_width(n)`	Specify the vector length (VL)
	`#pragma clang interleave(enable|disable)`	Enable/disable unrolling/interleaving of the loop
	`#pragma clang interleave_count(n)`	Specify how many iterations are to be unrolled/interleaved
Intel	`#pragma ivdep`	Ignore potential loop-carried dependencies
	`#pragma vector always`	Vectorize even if the estimated gain is low/negative
	`#pragma vector aligned`	Assume aligned data
	`#pragma vector unaligned`	Assume unaligned data

Tip: ***Compiler's manual***

Check the compiler's manual for comprehensive and updated descriptions of available vectorization pragmas and options.

Example: *Auto vectorizing a simple loop*

Here we perform a simple reduction of an array, with datatype `dtype` that is determined at compile time (`int` or `float`).

```
dtype sum_loop ( dtype *array, uint N )
{
    dtype sum = 0;
    for ( uint32_t i = 0; i < N; i++ ) {
        sum += array[i];
    }
    return sum;
}
```

Let's compile the code with GCC using different flags:

- To get the generated assembler in intel syntax: `-S -fverbose-asm -masm=intel`
- To enable vectorization and optimization: `-O3 -march=native -mtune=native -ftree-vectorize -funroll-loops`
- To get a report on vectorization: `-fopt-info-vec-optimized -fopt-info-vec-missed -fopt-info-loop`
- To set the data type: `-DTYPE=[INTEGER|FPDP]`

Now, inspecting the assembly code for the integer version:

```
.L4:
    vpaddd  ymm0, ymm0, YMMWORD PTR [rax]
    add     rax, 32
    cmp    rdx, rax
    jne    .L4
```

which means take what is inside register `rax` (the square brackets mean dereferencing), sum it with the value in `ymm0` and store the result in `ymm0` (all of 32 bytes, or 256 bits). Then increment `rax` by 32 bytes (the size of the vector) and repeat until `rax` reaches `rdx` (the end of the array). This is a clear example of vectorization: we are summing 8 integers at once, instead of one by one.

Vectorizing loops operating on integers works well, but for floating point numbers, we need to be careful.

```
.L4:
    vaddss  xmm0, xmm0, DWORD PTR [rax]
    add     rax, 32
    vaddss  xmm0, xmm0, DWORD PTR -28[rax]
    vaddss  xmm0, xmm0, DWORD PTR -24[rax]
    vaddss  xmm0, xmm0, DWORD PTR -20[rax]
    vaddss  xmm0, xmm0, DWORD PTR -16[rax]
    vaddss  xmm0, xmm0, DWORD PTR -12[rax]
    vaddss  xmm0, xmm0, DWORD PTR -8[rax]
    vaddss  xmm0, xmm0, DWORD PTR -4[rax]
    cmp     rax, rcx
    jne     .L4
```

Here, we are using the `vaddss` instruction which is actually a scalar instruction, thus we are not actually vectorizing the loop.

What we have just seen should convince you that integers can be easily vectorized by the compiler because there are no constraints on the reshuffling of the operations. Floating points, instead, don't have this advantage: even when some vectorization is possible, the critical paths that we insert through the semantics hinder the expression of data-level parallelism. To further test the result of the compilation, let's profile the codes using `perf` :

	Int	**Float**
Not vectorized	0 int_vec.add_256	0 vector 1,000,000 scalar
Vectorized	0 int_vec.add_256	250,000 vector 1,000,000 scalar

Of the $250,000$ int_vec.add_256, 125k are vector operations to initialize the loop and 12k are vector addition in the summation loop. For the float, 125k are to convert vectors of int to floats and 125k are to mov regs in memory. The $1,000,000$ scalar are not vectorized at all. Hence, autovectorization works nicely for integers but not at all for the floats. This descends from the compiler not being entitled to reshuffle operations in the summation loop. For as we have written it, $s+=a_i$, there is a clear critical path on s that must be respected by the compiler.

Observation: ***Critical path***

From the basic course of HPC you should remember that when trying to optimize a loop, if we are accumulating on a single variable (S) it may lead to the execution of a lot of bit reshuffling and scalar add, in place of clean vector operations. To fix this, we can use partial accumulators:

```
for (int i=0; i < N; i+=VL) {
    sum0 += array[i];
    sum1 += array[i+1];
    sum2 += array[i+2];
    sum3 += array[i+3];
    ...
}
sum = sum0 + sum1 + sum2 + sum3 + ...;
```

This exposes a natural vector pattern.

Now we can rewrite the loop differently, using multiple accumulators to break the critical path:

```
dtype sum_loop ( dtype * restrict array, const uint N )
{
    double sum0 = 0;
    double sum1 = 0;
    double sum2 = 0;
    double sum3 = 0;
    uint N4 = N\%0xFFFFFFFC;
    for ( uint32_t i = 0; i < N4; i+=4 ) {
```

```
            sum0 += array[i];
            sum1 += array[i+1];
            sum2 += array[i+2];
            sum3 += array[i+3];
        }
        ...
        return sum;
    }
```

This way we obtain:

	unroll vectorized	vectorized associative-math
metrics	500,000 vector 0 scalar	500,000 vector 0 scalar

Hence, re-engineering the code before using the auto-vectorization of the compiler can greatly enhance the achieved vectorization because we expose more parallelism while explicitly relaxing the order of the operations.

Example: *Another example*

Consider this loop:

```
for (int i = 0; i < N; i++) {
    sum += A[i]*B[i] + C[i];
}
```

This case has a higher possible parallelism since the reduction comes after the multiply-and-add vector operation. This means that a vector of partial results

```
D[i:i+n] = A[i:i+n]xB[i:i+n] + C[i:i+n];
```

can be preemptively prepared, waiting for the entries to be summed up serially. The compiler in fact opts for this implementation.

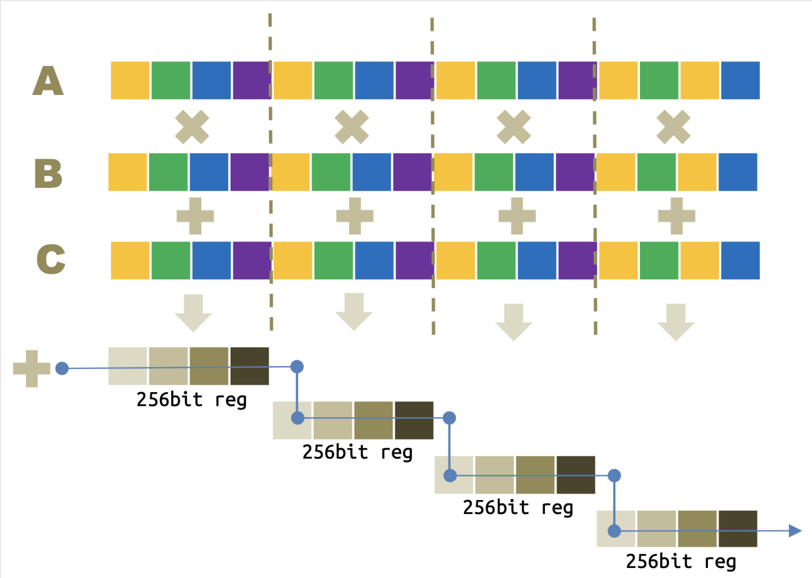


Figure 1.5: Auto-vectorization of a loop with multiply-and-add operation.

Lots of computation are performed via FMA on vector registers, 256b or 512b depending on what is available on the cpu. However, lots of reshuffling is needed and all the reduction summations are serial. Opting either for the usage of the `associative math` flag or for explicit loop unrolling (using separate accumulators) results in a different, fully vectorizable pattern.

So, to express the whole inherent loop-data and instruction-level parallelism is of primary importance to either

- authorizing the compiler to **re-shuffle floating-point** operations or
- **unroll the loops** to expose more instruction-level and data-level parallelism.

Tip:

Professor tip is to manually refactor the code, so that you can test for the correctness of the results while achieving the same vectorization.

The ideal situation is to have ***countable*** loops.

The iteration space must be known at run-time. The end of the loop can not depend on data. Following is an example of a non-countable loop:

```
int i = 0;
while (i < N) {
    a[i] = b[i] * c[i];
    if (a[i] > 0) {
        break;
    }
    i++;
}
```

Tip: *Countable While Loops*

Note that a countable while loop is vectorizable as a for loop is.

Moreover, there should not be branches, since the SIMD instructions are meant to perform exactly the same operation on multiple data and so different iterations can not have different control flows (if statements are admitted if they can result in a mask-like implementation). For the same reason, we should avoid function calls (vectorizable functions are an exception).

Sometimes it is essential to use specific compiler flags to enable or enhance vectorization of floating-point code, especially in the presence of math library calls or exception handling.

- **`-fno-math-errno`** : Tells the compiler to assume that math functions (like `sqrtf` , `logf` , etc.) do not set `errno` , and that your code will not check it. By disabling `errno` handling, the compiler is free to inline math functions or replace them with fast vector library calls, enabling vectorization even across calls to functions such as `libm` . This flag lifts a major legality blocker for SIMD.
- **`-fno-trapping-math`** : Informs the compiler that floating-point operations are not expected to raise traps (hardware FP exceptions) that your code will observe. This allows the compiler to speculate or execute floating-point operations in SIMD lanes that may not be used, enabling masked or predicated vectorization (e.g., by computing both sides of a branch and selecting the result).

By default, compilers like GCC have `-fmath-errno` and `-ftrapping-math` **enabled** (`ON`).

There are many situations where something could go wrong. The first case is when using **non-contiguous memory access**. It refers to loops that access data with a non-unit stride, or even worse with an irregular pattern. They are rarely vectorized, and that is because while consecutive data can be loaded in a single cache line, or in a register, with a single memory operation, sparse memory access need separate loads and data gathering; if the compiler estimates that this overhead is larger than the benefits, the loop is not vectorized. Another case is when there are **data dependencies** between iterations of the loop. If the compiler detects that there is a dependency, it can not vectorize the loop because it can not reshuffle the operations without changing the semantics of the code. For this case, we can identify:

- *Read-After-Write (flow-dependency)* $\rightarrow$ A variable is written in an iteration and read on a subsequent one. This loop can NOT be vectorized without leading to wrong results.
  However, let's consider the following 2D case:

  ```
  for (int i = 0; i < N; i++) {
      A[i][0] = C[i][0];
      for (int j = 0; j < M; j++) {
          A[i][j] = A[i][j-1] + C[i][j];
      }
  }
  ```

  the dependency is carried in the inner loop, while the outer loop iterations do not exhibit any dependency and the **outer loop vectorization** can be performed.

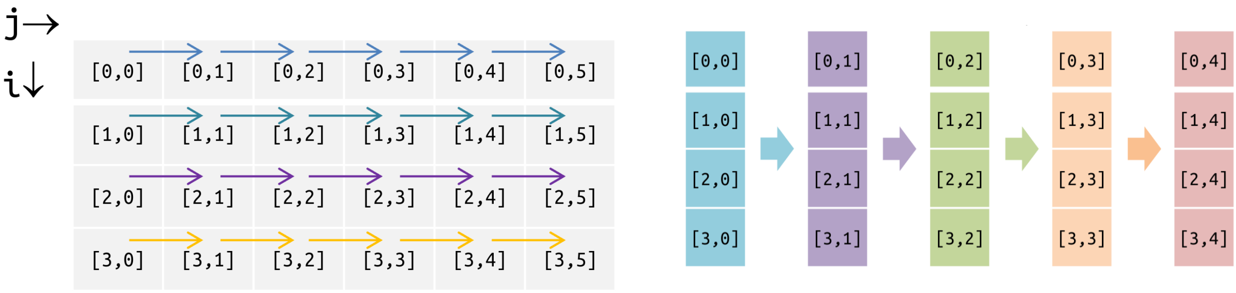


Figure 1.6: RAW data dependency: 4 (or 8) outer iterations can be executed together because there are no vertical dependencies (outer loop vectorization)

Consider the dependency graph of the iteration space. It clearly shows that there are no vertical dependencies. Hence, the outer loop can be vectorized: 4 (or 8) outer iterations can be executed together because there are no vertical dependencies. However, if the elements are stored in memory in row-major order that require a lot of memory gather and scatter operations. If not supported in hw, they must be emulated, which is even more expensive. Thus, we may consider to write the data column-major if possible.

- *Write-After-Read (anti-dependency)* → A variable is read in one iteration and written in a subsequent one. Unsafe for parallelism (no strict order among iterations assigned to tasks), normally safe for vectorization. Consider this example:

```
for (int i = 1; i < N; i++) {
    a[i-1] = a[i] + c[i];
}
```

while the following cannot be vectorized because it's unclear if `a[i]` may be overwritten before the summation to `sum` :

```
for (int i = 0; i < N; i++) {
    a[i-1] = a[i] + c[i];
    sum += a[i];
}
```

- *Write-After-Write (output dependency)* → The same variable is written in more than one iteration. Generally unsafe both in parallelization and in vectorization.

```
double sum = 0;
for (int i = 1; i < N; i++) {
    a[i-1] = function1(i);
    a[i] = function2(i);
}
```

- *Memory ambiguity* → Typically due to memory aliasing. It may be impossible for the compiler to decide whether, for some value of `i`, `a[i]`, `b[i]` and `c[i]` will somehow overlap. If that is the case, it will not issue a vector code. When it is possible, the compiler will issue more than one version of the loop and a code that performs an overlap test among the pointers. Then the execution will be routed to the correct version at run-time.

```c
void function (int *a, int *b, int *c, int N) {
    for (int i = 0; i < N; i++) {
        c[i] = a[i] + b[i];
    }
}
```

We can disambiguate using the `restrict` qualifier, which tells the compiler that the pointers do not alias each other. This allows the compiler to vectorize the loop without generating multiple versions and runtime checks. Also the `const` qualifier can help, as it tells the compiler that the data pointed to by the pointer will not be modified, which can also enable vectorization.

A variable is said to be **aligned** in memory when its first byte's address is a multiple of the variable's type size:

- `short int` $\rightarrow$ 2 bytes: 0, 2, 4, 6, 8, ... are aligned placements;
- `int / float` $\rightarrow$ 4 bytes: 0, 4, 8, 12, ... are aligned placements;
- `long long / double` $\rightarrow$ 8 bytes: 0, 8, 16, 24, ... are aligned placements.

Consequently, an aligned variable is also aligned w.r.t. the cache line it belongs to: it means that an aligned variable will never cross 2 cache lines! However, let's imagine that we allocate a bunch of memory and we consider it as an array of elements with byte size s_B. Let's say that:

- its starting byte is aligned at a_B bytes (a_B is a multiple of s_B);
- we access this array by vectors of element-size V_L (meaning that every vector has V_L elements); hence we will access $V_L \times s_B$ bits every time.

Then if

$$(a_B + V_L \times s_B)\%C_L == 0 \quad (C_L \text{ being the cache capacity in terms of elements})$$

all the vectors will stay in the same cache line and will require 1 load; otherwise, a vector will be mapped into two diffent cache lines and will require 2 loads.

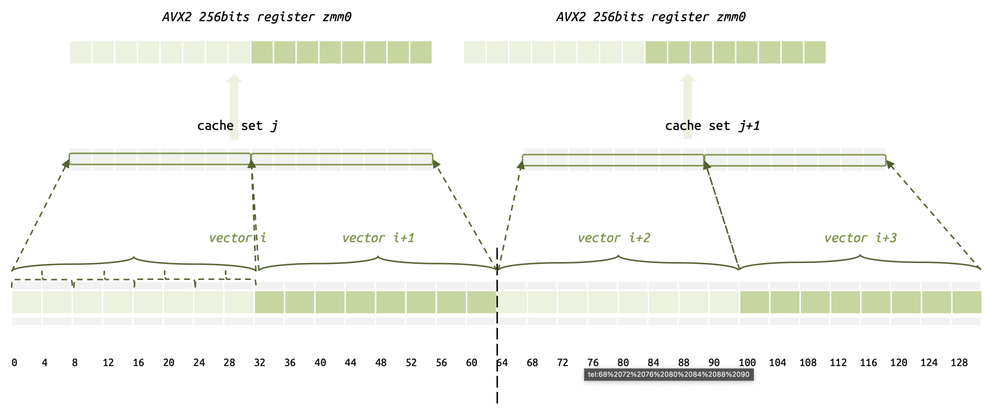


Figure 1.7: Example 1: an array of float aligned at 64 bits accessed as vector of size 4 (cache capacity is 16 with a std cache line of 64B) $a_B = 8B, V_L = 4, s_B = 4, C_L = 16$ every vector will occupy exactly 1 cache line.

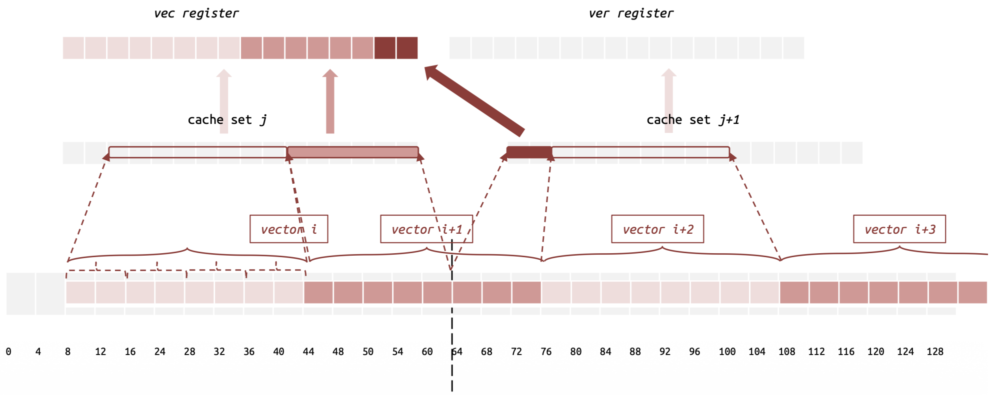


Figure 1.8: Example 2: an array of float aligned at 16 bits accessed as vector of size 4 (cache capacity is 16 with a std cache line of 64B) $a_B = 8B, V_L = 4, s_B = 4, C_L = 16$ every vector will occupy exactly 2 cache lines.

Tip: ***Data structures***

- Keep together data that are used together in vector operations;
- The smallest the data type you use, the larger the number of data that you can fit in the cache and in the vector registers;
- Do not mix same-type different-size data:

```
func ( float *A, float *B, double *C, int N ) {
    for ( int i = 0; i < N; i++ ) {
        A[i] = B[i] * C[i];
    }
}
```

the type conversion renders things less efficient (`long double` type is not supported on GPUs).

Very commonly the data are multi-dimensional and hence the most natural representation is by a structure that encapsulates all the quantities for a single data point. Every entry of the array that contains all the data points is then a structure:

```
data[i].pos[3], data[i].acc[3], data[i].mass, ...
```

This is the **Array of Structures (AoS)** representation. and is a very convenient representation in many respects. However, when the structures collects many and diverse data, the SoA have several inconvenience too when computational performance is considered.

Let's consider the following example that occupies 96 bytes:

```
struct particle {
        double pos[3];
        double acc[3];
        double mass;
        ...
    };
```

As a first consideration, even a single particle could not fit in a cache line. Second, most probably pos[3] will be used, likely with mass, to get accel[3] and then vel[3] will be updated.

Very likely every of these calculations may have a high degree of vectorization. However, there is no way in which those quantities can be loaded from memory to vector register efficiently without lots of overhead, either using gathering instructions or allocating additional memory and moving there only the variables used for every calculation.

So, a very first step consist in opting for **Structures of Arrays of (small)Structures** (SoAoS):

```
struct Ppos       { double pos[3], double mass; };  Ppos *mpositions;
struct Paccel     { double accel[3]; };             Paccel *
    accelerations;
struct Pvel       { double vel[3]; };               Pvel *velocities;
struct Pchemistry {float elements[n_el];};          Pelems *Pchemistry;
                                                    long long int *PIds;
```

Clearly, that alleviates the aforementioned issues but it is still sub-optimal.

If, instead, we opt for a **Structure of Arrays** (SoA) approach:

```
double *x, *y, *z, *mass, *xacc, *yacc, *zacc, ...
long long *ids;
```

the possible vectorization is definitely higher. Data rearranged in SoA approach definitely would expose more, and more efficient, vectorization opportunities.

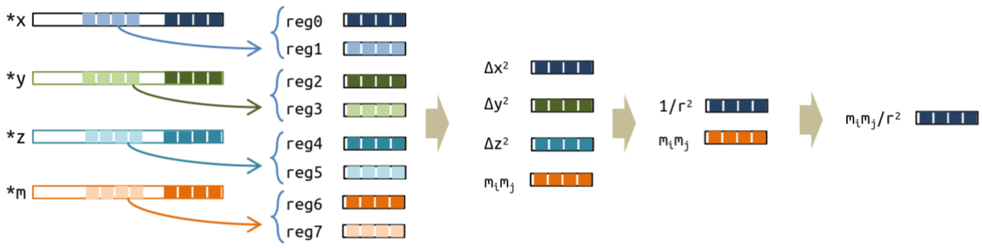


Figure 1.9: AoS vs SoA

⚠ Warning:

What is really convenient in every specific case may depend on several additional factors and that, in the context of this lecture, hinders more than just mentioning and explaining the vectorization advantage of SoA.

1.5 Vector Types

It is possible to use a vector type (not present in standard C, but commonly accepted by compilers) to declare a vector of a given type.

```
typedef type name __attribute__((vector_size,(bytes)));
```

where:

- **type**: the basic type for every element of the vector.

- **name**: the name with which you will refer to the type in your code.
- **bytes**: the number of bytes of the vector.

```c
/* Declare a vector of 4 doubles */
typedef double v4d __attribute__((vector_size,(4*sizeof(double))));
/* Declare a vector of 8 doubles */
typedef double v8d __attribute__((vector_size,(8*sizeof(double))));
/* Declare a vector of 4 integers */
typedef int v4i __attribute__((vector_size,(4*sizeof(int))));
```

It is not possible to access the elements of a vector type directly, in the same way as you cannot access a single byte of a double. However, there is a standard way to do that:

```c
typedef union {
    v4d v;
    double d[VD_SIZE];
} v4d_u;
```

1.5.1 Memory Alignment

It is mandatory that memory regions accessed by vector instructions are aligned (32B for AVX/AVX2 and 64B for AVX-512).

Loops translated by the compiler consist of three sections:

1. **Peeling loop** - scalar until aligned addresses are reached ($\rightarrow$ avoided by proper alignment);
2. **Main body** - well vectorized from aligned addresses;
3. **Remainder loop** - scalar operations if the number of elements is not a multiple of the VL ($\rightarrow$ maybe avoided by hinting the compiler like `__attribute__((__assume(N%4==0)))` or by accounting for the main body and the remainder explicitly).

To allocate memory in an aligned way, we can use C11 and POSIX functions:

- **C11**: `void *aligned_alloc(size_t alignment, size_t size);`
- **POSIX**: `void *posix_memalign(void **memptr, size_t alignment, size_t size);`

It is also possible to use a static allocation (i.e. variables or automatic arrays):

```c
__attribute__((aligned(base))) <var>;
```

And it is also possible to assume that the memory is aligned:

```c
assume_aligned(<array>, base);
```

Observation: ***Unrolling loops by hands***

When unrolling a loop by hands, which you typically do in order to expose more parallelism as we have discussed previously, you also need to separately handle the main loop and the remind loop.

```c
for (int i = 0; i < N; i++) ...
```

becomes:

```
int N4 = (N/4)*4;    // --> integer division ensures that N4 is the
                     // largest multiple of 4 smaller than N
for (int i = 0; i < N4; i+=4) ... // main body

for (int i = N4; i < N; i++) ... // remainder loop
```

You can give some additional hint to the compiler so that to avoid the issuing of additional codes to check the peeling. The line

```
int N4 = (N/4)*4;
```

can be replaced by

```
N4 = N&0xFFFFFFFC;   // --> use 0xFFFFFFF8 for 8
                     // --> use 0xFFFFFFFFFFFFFFFF<C|8> for 64 bits integers
```

this will hint the compiler that N4 is a multiple of 4, without need of issuing additional code. Alternatively,

```
N4 __attribute((__assume(N4\%4==0)));
```

may also work on some compilers.

Example: ***Working with vectors***

Let's re-implement the following loop using the vector types and check what changes in the generated assembler code and in the run-time.

```
double kernel( double *restrict A, double *restrict B, double *restrict C, int N ) {
    double sum = 0;
    for ( int i = 0; i < N; i++ )
        sum += A[i]*B[i] + C[i];
    return sum;
}
```

At first we need to define the appropriate vector variables:

```c
#define VD_SIZE 4
typedef double v4df __attribute__((vector_size(VD_SIZE*sizeof(double))));
typedef union {
    v4df v;
    double d[VD_SIZE];
} v4df_u;

v4df *VA = (v4df *) A;
v4df *VB = (v4df *) B;
v4df *VC = (v4df *) C;

v4df vsum = {0};
```

Now we can actually implement the sum:

```c
int N4 = N&0xFFFFFFFC;

for (int i = 0; i < N4; i++) {
    vsum += VA[i] * VB[i] + VC[i];
}

v4df_u *vsum_u = (v4df_u *) &vsum;
vsum_u->v[0] += vsum_u->v[1] + (vsum_u->v[2] + vsum_u->v[3]);

for (int i = N4; i < N; i++) {
    sum += A[i]*B[i] + C[i];
}

sum += vsum_u->v[0];

return sum;
```

1.5.2 Conditional evaluation

When our code contains conditional statements, we need to pay special attention if we want to use vector types. Vectorization is usually only possible if the condition is simple and the code inside the conditional is not too complex, for example a single operation or a small sequence of straightforward instructions.

```
void kernel( double * restrict A, double * restrict B, const int N ) {
    for ( int i = 0; i < N; i++ )
        if ( A[i] < B[i] ) swap(A, B);
    return;
}
```

Actually this code can be easily fully vectorized by the compiler itself with the appropriate compilation flags. The idea is to create a mask of the condition and then use it to select the appropriate values.

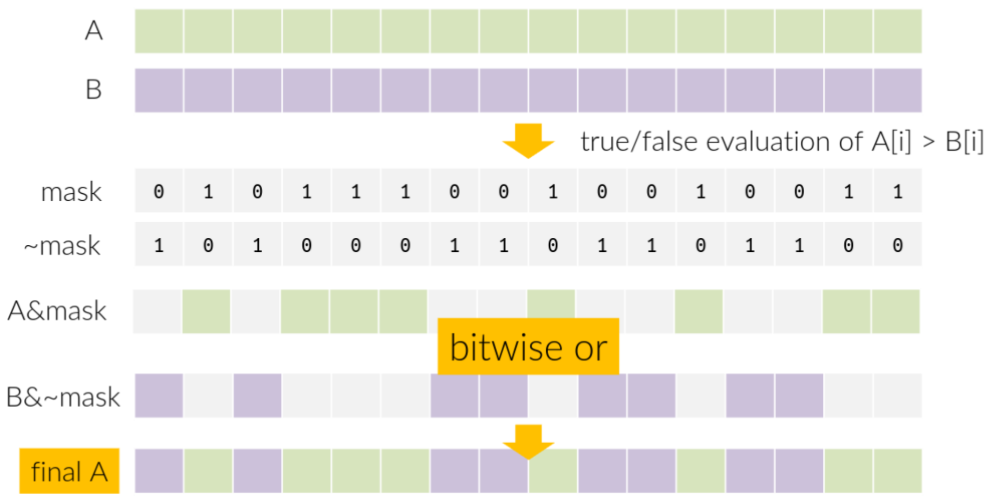


Figure 1.10: Conditional evaluation [**ADV_HPC**]

Tip:

The concept of vectorized conditional flow can really be thought as ”projecting” througha mask so that the final values that are ”impressed” in the memory positions pass through the ”transparent” entries and are blocked by ”opaque” ones. Many projected values could of course overlap by using multiple masks combined with bit-wise operators.

To efficiently handle conditionals in vectorized code, we first implement the mask logic. The ternary operator is particularly well-suited for expressing such element-wise conditional assignments:

```
void kernel( double *A, double *B, int N ) {
    // element-wise swap of A and B so that
    // A[i] > B[i] for every i

    for ( int i = 0; i < N; i++ ) {
        double min = (A[i]>B[i] ? B[i] : A[i]);
        double max = (A[i]>=B[i] ? A[i] : B[i]);
        A[i] = max;
```

```
        B[i] = min;
    }
    return;
}
```

Now we can vectorize the code using the vector types and the mask logic.

```
dvector_t *vA = (dvector_t *)__builtin_assume_aligned(A, VALIGN);
dvector_t *vB = (dvector_t *)__builtin_assume_aligned(B, VALIGN);

int VN = (N/DVSIZE)&0xFFFFFFFC;
IVDEP
LOOP_VECTORIZE
LOOP_UNROLL_N(4)
for ( int i = 0; i < VN; i++ ) {
    dvector_t a = vA[i];
    dvector_t b = vB[i];
    llvector_t keep = (vA[i]>=vB[i]);
    vA[i] = (dvector_t)(((llvector_t)a & keep) |
            ((llvector_t)b & ~keep));
    vB[i] = (dvector_t)(((llvector_t)b & keep) |
            ((llvector_t)a & ~keep));
}

int j = VN*DVSIZE;
process ( &A[j], &B[j], N-j+1);
```

TODO: Add second version and possibly comment

1.6 Vectorization by OpenMP SIMD directive

The OpenMP compiler may transform a loop that is marked with the simd construct into a SIMD loop. As a result, multiple iterations of the loop may be executed concurrently by a single thread. A SIMD loop of length n consists of the logical iterations $0, 1, \ldots, n-1$. The numbering denotes the sequential order in which loop iterations are executed.

A SIMD *chunk* refers to the set of iterations that are executed concurrently by a SIMD instruction in a single thread. Within a chunk, each iteration is executed by a SIMD *lane*, which refers to the mechanism that a SIMD instruction uses to process one data element. The number of iterations in a SIMD chunk is the vector length.

The OpenMP specification describes the execution model of the simd construct as follows: ”When an OpenMP thread encounters a `simd` construct, the iterations of the loop asociated with the construct may be executed concurrently using the SIMD lanes that are available to the thread.” Intentionally, this allows for much freedom in the implementation.

```
#pragma omp simd [clause [[,] clause] ...]
```

The clauses are:

- `private(list)` and `lastprivate(list)`: they have exatly the same meaning as in “thread-related” OpenMP. The execution instances in this context are the SIMD lanes; hence, an instance of every private variables is created per SIMD lane.

```c
int i;
double tmp, sum=0;

#pragma omp simd private(tmp) reduction(+:sum)
for (int i = 0; i < N; i++) {
    tmp = function(b[i]);
    array[i] = tmp;
    sum += tmp;
}

// NOTE: i is lastprivate
```

- `reduction(identifier: list)` and `collapse(n)`: they also retain their usual meaning and usage.

```c
int i;
double sum=0;

#pragma omp simd reduction(+:sum)
for (int i = 0; i < N; i++) {
    for (int j = 0; j < M; j++) {
        tmp = function(a[i], b[j]);
        array[i] = tmp;
        sum += tmp;
    }
}
```

- `simdlen(length)`: It has the goal of suggesting the element-size of the vector lane to be adopted, i.e., the count of elements in the vector.

```c
int i;
double tmp;

#pragma omp simd simdlen(32/sizeof(double))
for (int i = 0; i < N; i++) {
    tmp = function(b[i]);
    array[i] = tmp;
}

// NOTE: i is lastprivate
```

- `safelen(length)`: It is used to specify at the compiler the maximum length not having dependences inside the loop. In the following example, we instruct the compiler that 8 is a safe length for vectorization;

```c
#pragma omp simd safelen(8)
for (int i = 0; i < N; i++) {
    a[i] = pow(b[i-k], 3.14); // here the compiler does not know a
        priori
}
```

- `linear(list[: linear-step])`;
- `aligned(list[ : alignment ])`;

1.6.1 The omp SIMD functions

In general, every non-linear element in the execution flow is at high risk a disruption in the vectorization. Function call above all, but also, for instance, conditionals.

OpenMP SIMD offers the possibility of automatically build vector version of code segments through the `declare simd` directive. These functions can then be called from a simd loop.

```
#pragma omp declare simd
double foo (double x, double y, double z) {
    double res = x*y + z;
    return res;
}

void loop (double *a, double *b, double *c, int N){
    #pragma omp simd
    for (int i = 0; i < N; i+=4) {
        a[i] = foo(a[i], b[i], c[i]);
    }
}
```

The `declare simd` directive instructs the compiler to generate at least one additional simd version of the function `foo` because it will be called from a simd loop. That variant will be called from inside the simd loop in the function `loop`.

However, when declaring `foo`, can we give the compiler more details about how x, y and z are changing? In this form, the best assumption is that they must be incremented by 1 in the simd version.

The possible clauses are:

- `uniform`: when listed as uniform, a parameter is intended to be invariant for all the concurrent calls to the function, i.e. all the simd lanes will observe the same value
- `linear`: at odds, being linear means that a parameter changes linearly with the indicated step
- `simdlen`: retains the same meaning as in the simd directive
- `aligned`: as exactly the meaning than in normal code
- `inbranch`: informs the compiler that the function will be called from in a conditional branch.
- `notinbranch`: informs the compiler that the function will not be called from in a conditional branch.

TODO: add examples

1.6.2 The omp SIMD conditionals

OpenMP simd offers two additional clauses that work as assertions: `inbranch` and `notinbranch`. The former informs the compiler that the function will be called from within a conditional branch, whereas the last guarantees the compiler that this will not happen. The compiler can then opt for very different optimizations and how to manage the vectorization.

```
#pragma omp declare simd inbranch
float do_mult(float x) {
    return (-2.0*x);
}
```

```c

#pragma omp declare simd notinbranch
extern float do_pow(float);

void simd_loop_with_branch(float *a, float *b, int n) {
    #pragma omp simd
    for (int i = 0; i < n; i++) {
        if (a[i] < 0.0) {
            b[i] = do_mult(a[i]);
        }
        b[i] = do_pow(b[i]);
    } // end simd region
}
```

Finally, multiple simd versions of the same function can be generated with multiple declarations:

- `#pragma omp declare simd linear(pixel) uniform(mask) inbranch`;
- `#pragma omp declare simd linear(pixel) notinbranch`;
- `#pragma omp declare simd`;
- `extern void compute_pixel(char *pixel, char mask)`;

2
OpenMP

2.1 Tasks

A **task** in OpenMP is an independent unit of work, comprising a well-defined set of instructions that can be executed asynchronously, enabling the decomposition of complex computations into smaller pieces that can be scheduled and executed in parallel by different threads. This **task abstraction** allows for flexible parallelism, supporting dynamic workloads with unpredictable execution flows.

Data Abstraction refers to the encapsulation of logically uniform pieces of information (arrays, structures, ...) that may be accessed concurrently by multiple threads. Proper management of data abstraction is essential to avoid race conditions and ensure consistency, especially when multiple tasks access shared data out-of-order. Mechanisms such as data dependencies, synchronization constructs, and memory models are employed to manage safe and efficient concurrent access.

Tasks often interact through shared data, giving rise to dependencies. They form a **task dependency graph**, where nodes represent tasks and directed edges represent data dependencies (i.e., a task must wait for another due to shared data). It is crucial that this graph is ***acyclic***, ensuring there are no circular wait conditions or deadlocks. It is sometimes possible to parallelize a workflow that is irregular or depends on runtime conditions using ***sections***. However, these solutions often become difficult to manage, and the intrinsic rigidity of the sections construct makes it difficult to express dynamic patterns of parallelism.

Since version 3.0, OpenMP introduced the **task** construct, which provides a flexible way to handle irregular execution flow and dynamic task creation. When defining a task, OpenMP internally creates a ”unit of work”, bundling together all necessary code, data, and local variables. This work is then scheduled for execution at some future point. Behind the scenes, OpenMP employs a queuing system to orchestrate the assignment of these tasks to the available threads, balancing the workload and enabling dynamic parallelism. A main thread generates tasks and a pool of threads executes them. As soon as they are free, the threads pick up one queued task each and execute it. When the main thread finishes creating tasks, it picks up a queued task and executes it as well.

Observation: ***Tasks vs Threads***

Creating tasks does not mean creating threads. The tasks are “units of work” that are assigned to the running threads. The pool of threads is unaltered (unless nested parallelism is involved) and mapped onto the underlying physical cores.

Task management in OpenMP revolves around three key concepts to take into account for writing correct and efficient parallel code with OpenMP tasks:

- **Data environment**: how data is assigned to each task, distinguishing shared and private variables.
- **Creation and execution**: when and how many tasks are generated, who executes them, and whether prioritization applies.
- **Synchronization and dependence**: how tasks are coordinated, scheduled, and whether they depend on other tasks' completion.

2.1.1 Creating tasks

Let's start with a simple example, where we have a single region with a single thread that creates two tasks and then waits for all the other threads to finish. Once the tasks are created, they become available to be executed by the other available threads.

```
#pragma omp parallel
{
    #pragma omp single // a single thread creates the tasks
    {
        printf( "Yuk yuk, here is thread %d from "
            "within the single region\n", omp_get_thread_num() );
        #pragma omp task
        {
            printf( "\tHi, here is thread %d "
                "running task A\n", omp_get_thread_num() );
        }
        #pragma omp task
        {
            printf( "\tHi, here is thread %d "
                "running task B\n", omp_get_thread_num() );
        }
    } // all the other threads waits here
    printf(" :Here is thread %d after the single region, I was"
        "waiting for all the others\n", omp_get_thread_num());
}   // end of the parallel regions  \
```

Running this multiple times shows that a random thread enters the single region while others wait at the barrier. Some waiting threads will pick up the generated tasks. Once all tasks are completed, every thread proceeds.

```
Yuk yuk, here is thread 5 from within the single region
   Hi, here is thread 2 running task A
   Hi, here is thread 1 running task B
:Hi, here is thread 5 after the single region, I was waiting for all the others
:Hi, here is thread 1 after the single region, I was waiting for all the others
:Hi, here is thread 0 after the single region, I was waiting for all the others
:Hi, here is thread 6 after the single region, I was waiting for all the others
:Hi, here is thread 4 after the single region, I was waiting for all the others
:Hi, here is thread 3 after the single region, I was waiting for all the others
:Hi, here is thread 7 after the single region, I was waiting for all the others
:Hi, here is thread 2 after the single region, I was waiting for all the others
```

Nowait

If we add a `nowait` clause to the single directive, we will get that the greeting message from all the other threads will almost certainly arrive before the greetings from the tasks execution.

```
#pragma omp parallel
{
    #pragma omp single nowait
    {
    ...
```

In fact, the `nowait` lets the threads skip the single region entirely, and execute what is next, until the first scheduling point; which now is the very end of the parallel region.

Taskwait

Adding a `taskwait` directive at the end of the single region introduces an explicit synchronization point: the thread executing the single region will suspend at that point until all tasks it directly created have completed. Only then proceeds to subsequent statements within the single block.

```c
#pragma omp parallel
{
    #pragma omp single nowait
    {
        #pragma omp task
        { /* Tasks A, B, ... */ }

        #pragma omp taskwait  // wait for A and B to complete
        printf("All tasks have been executed\n");
    }
    printf(" :Here is thread %d after the single region\n",
        omp_get_thread_num());
}
```

The print following `taskwait` executes only after all tasks have finished. Meanwhile, the `nowait` clause allows other threads to proceed and print their messages independently of task completion.

```
 :Here is thread 3 after the single region
 :Here is thread 4 after the single region
 :Here is thread 2 after the single region
 :Here is thread 6 after the single region
 :Here is thread 7 after the single region
 :Here is thread 0 after the single region
 :Here is thread 1 after the single region
   Hi, here is thread 7 running task A
   Hi, here is thread 4 running task B
 All tasks have been executed
 :Here is thread 5 after the single region
```

⚠ Warning: ***Shallow synchronization***

The `taskwait` directive waits only for tasks directly created within the enclosing task region. If Task A itself creates Task A1, the `taskwait` will return as soon as A completes, even if A1 is still running. For deep synchronization that includes all descendants, use `taskgroup`.

Task scheduling points

A **task scheduling point** is when the runtime may suspend the current task and switch to another; the choice among available tasks is implementation-defined. Scheduling points occur during and after explicit task generation, after task completion, within `taskyield` regions, and inside synchronization constructs:

- **barrier** (implicit or explicit): All tasks created by any thread in the current `parallel` region are guaranteed to be complete when the barrier exits. Implicit barriers occur at the end of `parallel`, `for`, `sections`, and `single` (without `nowait`).
- **taskwait**: The encountering task suspends until all its *direct child* tasks have completed; it does *not* wait for descendants.
- **taskgroup**: All tasks generated within the region, *including their descendants*, are guaranteed to complete before the region exits.

2.1.2 Scope of variables

A key consideration when working with tasks is that a task may be executed at an arbitrary time after its creation, potentially by a different thread. The data environment of the task (which variables it can access and how) must therefore be determined at the moment of task creation, not execution.

Data-sharing clauses

- `shared()` : All accesses inside the task refer to the *same* storage as the original variable in the enclosing context. No copy is made; changes are visible to other tasks/threads.
- `private()` : Each task gets its own *uninitialized* copy of the variable. The original is unaffected.
- `firstprivate()` : Each task gets its own private copy, *initialized* with the value the variable had at task creation time. This is the safest choice for local variables whose value the task needs.

> **Warning: *Common pitfall***
>
> If a task uses a `shared` reference to a local variable, and it goes out of scope before the task executes, the behavior is undefined. Use `firstprivate` to capture the value safely.

Implicit data-sharing rules for tasks

The rule-of-thumb is: *data, unless otherwise stated, are copied into local copies to preserve the data context at the moment of task creation*. The effect is the same as `firstprivate`.

- If a variable is **shared** in all enclosing constructs up to and including the innermost enclosing `parallel` region, then it is **shared** in the task.
- Otherwise, the variable is implicitly **firstprivate**.
- Variables with automatic storage declared inside the task region are **private**.
- Global and static variables are **shared** (predetermined).
- Dynamically allocated memory (heap) is **shared**.

Tasks need to capture the values of local variables at creation time (since may go out of scope or change before the task runs), while truly shared data should remain accessible to all tasks.

```c
double pi = 3.1415;
double a = 0.0;

#pragma omp parallel
{
    // pi, a : shared
    #pragma omp single private (a)
    {
        int 1 = 1;
        // a : private but not initialized
        // i : private because it's a local variable
        #pragma omp task
        {
            int j = 0;
            // i, a : firstprivate by default (value is undefined)
            // j : is a task's private variable
            // pi : shared
        }
    }
}
```

Tied and untied tasks

By default, tasks are **tied**: once a thread begins executing a tied task, only that same thread can resume it if the task is suspended. This simplifies reasoning about thread-local state.

With the `untied` clause, a suspended task may be resumed by *any* available thread in the team:

```
#pragma omp task untied
{ // This task can migrate between threads }
```

Untied tasks offer more flexibility and can improve load balancing, but with some caveats:

- `threadprivate` variables must not be used (the executing thread may change).
- The value of `omp_get_thread_num()` may change after a scheduling point within the task.
- Thread-local resources (e.g., thread-specific allocators) need to be handled carefully.

2.1.3 Tasks synchronization

A fundamental consideration in asynchronous task execution is determining *when* tasks actually run and how to properly coordinate them. Upon creation, a task may either be **deferred** (scheduled for later execution while the generating task region is suspended) or executed immediately. This flexibility introduces synchronization challenges: not all techniques that work well for thread-based parallelism apply directly to task-based execution.

In particular, **mutual exclusion** mechanisms (critical sections, atomic operations, locks) remain valid for tasks, since they protect shared data regardless of which thread or task accesses it. However, **event synchronization** constructs (barriers, busy-wait loops, boolean flags) are problematic: because tasks may be deferred or migrated, waiting for an event that depends on another task's progress can easily lead to deadlock if the waiting thread is the one that would otherwise execute the awaited task.

Synchronization constructs

We have already seen that constructs such as `taskwait`, `taskgroup` and `barrier` can be used to synchronize tasks.

- `barrier`: A barrier synchronizes all threads in the current team. All threads must reach the barrier before any can proceed. Note that barriers apply to threads, not tasks: using a barrier inside a task can lead to deadlock if not all threads reach it.
- `nowait`: This clause removes the implicit barrier at the end of worksharing constructs (such as `single`, `for`, `sections`). When applied to a `single` construct that generates tasks, other threads skip the single region and continue execution. Note that `nowait` does *not* wait for child tasks to complete; it only removes the implicit barrier for the worksharing construct itself.
- `taskwait`: This directive causes the encountering task to wait until all its *direct* child tasks have completed. It binds to the current task region, and the set of binding threads is the current team. When a thread encounters a `taskwait` construct, the current task region is suspended until all child tasks generated before the `taskwait` complete execution. Note that `taskwait` does *not* wait for descendants of child tasks (i.e., grandchildren).
- `taskgroup`: This construct provides more sophisticated synchronization by waiting for all *descendant* tasks (children, grandchildren, etc.) created within the taskgroup region to complete. This is useful for complex workflows with nested task generation.

2.1.4 Memory consistency model

Sequential Consistency

Consider the following scenario: we have two shared variables, $A = 0$ and $B = 0$, and two threads:

Thread a

```
[a1] A = 1;
[a2] print(B);
```

Thread b

```
[b1] B = 1;
[b2] print(A);
```

One may ask: what values can be printed for A and B by the respective threads? To answer this, let's introduce **Sequential Consistency**, a memory model defined by Leslie Lamport in 1979.

Sequential consistency means that the results of execution are the same as if all operations from all threads were executed in some sequential order, and each thread's operations appear in this sequence in the order written in the code. Put simply, every individual thread issues memory operations in program order, and all such operations appear as though they are executed one at a time in a global sequence.

In our example, under sequential consistency, the possible outputs printed by each thread for A and B are restricted by the orderings of the two stores and loads. For instance, it won't be possible for both threads to print 0, since that would imply that both the write to A and the write to B happened after both reads, which contradicts a single global ordering. Instead, possible outputs include (A=0, B=1), (A=1, B=0), or (A=1, B=1), depending on how the stores and loads are interleaved. Sequential consistency guarantees that the memory operations will be observed in an order that makes intuitive sense, as if there were some global sequence in which all operations happened.

Relaxed Consistency

Let's consider now the following scenario:

Thread a

```
[a1] A = 1
```

Thread b

```
[b1] A = 2
```

Thread c

```
[c1] print A
```

Even in the presence of local memory, which is permitted by OpenMP's memory model, the cache coherency protocol guarantees that writes to the same memory location are observed by all threads in the same order, regardless of what that order is. Under sequential consistency, either [a1] or [b1] must appear first in the global memory order, and all threads will observe that same ordering.

The drawback of sequential consistency is performance: it effectively serializes memory operations, preventing many hardware optimizations. For this reason, modern systems implement **relaxed memory models** that are better suited to out-of-order CPUs with deep cache hierarchies.

Under a relaxed memory model, threads may buffer writes to local storage (such as cache memory) before propagating them to main memory. Consider again our first example with threads a and b: thread a writes to A and reads B, while thread b writes to B and reads A. The key insight is that there is no ordering constraint between thread a's write to A and thread b's read of A, nor between thread b's write to B and thread a's read of B.

This means both threads could buffer their writes locally and issue their reads before those writes become visible to other threads. Without explicit synchronization, each thread reads from main memory (or its private cache), where both A and B are still 0. The result: both threads print 0, an outcome that sequential consistency would forbid.

OpenMP memory model

To understand how OpenMP handles these complexities, its memory model is defined in terms of two key concepts:

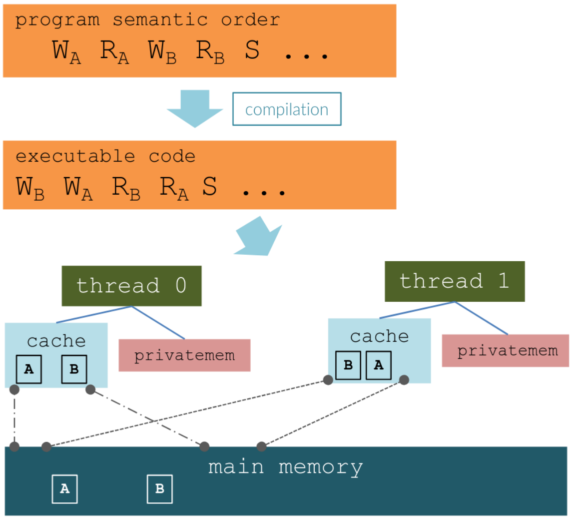
- **Coherence**: the prescribed behavior of the memory system when multiple accesses to the same memory location are performed by different threads.
- **Consistency**: the order in which memory accesses by different threads become visible to one another.

These concepts work together to specify when and how memory updates propagate between threads.

The OpenMP memory model is a relaxed-consistency, shared-memory model. All threads share access to a common memory space where variables can be stored and retrieved. However, each thread is also permitted to maintain its own **temporary view** of memory, which may reside in registers, cache, or other local storage. This temporary view allows threads to avoid accessing main memory on every variable reference, improving performance. Importantly, a thread's temporary view can become inconsistent with main memory or with other threads' views until an explicit synchronization occurs, typically via `flush`, barriers, or other synchronization constructs that force the views to reconcile. In addition to shared memory, each thread has access to **threadprivate memory**, a private storage area that cannot be accessed by other threads.

In practice, the relaxed memory model introduces several layers of reordering between what the programmer writes and what actually happens in memory:

1. **Compiler reordering**:
   The compiler may rearrange the source code's operations into any semantically equivalent order that it deems more efficient.
2. **Hardware reordering**:
   The processor may further reorder instructions at runtime, so the actual memory commit order can differ from the compiled code order.
3. **Temporary views**:
   Each thread's temporary view of memory may diverge from main memory (and from other threads' views) until synchronization occurs.



The consistency model specifies which orderings between reads (R), writes (W), and synchronizations (S) are guaranteed. OpenMP enforces that all S operations (barriers, flushes, locks, etc.) appear in the same order across threads, giving a sequentially consistent view for synchronization events. Within each thread, the following orderings are preserved: $R \to R$, $W \to W$, $R \to W$, $S \to R$, $S \to W$, $R \to S$, $W \to S$, and $S \to S$. Notably, $W \to R$ is *not* guaranteed, meaning a read may be reordered before a preceding write, so only synchronization points prevent stale data from being observed.

Recall that under sequential consistency, all memory operations appear to execute in some global order that respects each thread's program order, and all threads observe the same ordering. In this case, the program order, code order, and commit order are effectively equivalent. Relaxed consistency lifts some of these constraints: threads may observe memory updates in different orders, and only synchronization points enforce a consistent view across threads to ensure safe, predictable parallel execution.

2.1.5 FLUSH operations

OpenMP `flush` operations are used to enforce the consistency between a thread's temporary view of the main memory and the main memory itself, or between multiple threads' temporary views.

Strong FLUSH

A strong flush operation enforces consistency between a thread's temporary view and main memory. It is applied to a specified set of variables called the **flush-set**. When a strong flush is executed, the implementation must not reorder memory operations for variables in the flush-set with respect to the flush itself; this restriction applies both to the code generated for memory operations and to the code for the flush operation on those variables.

The primary purpose of a strong flush is to ***guarantee that a value written to a variable by one thread may be read by a second thread***. To achieve this, the programmer must ensure that the second thread has not written to the variable since its last strong flush, and that the following sequence of events occurs in this specific order:

1. Thread 1 writes the value to the variable.
2. Thread 1 performs a strong flush of the variable.
3. Thread 2 performs a strong flush of the variable.
4. Thread 2 reads the value from the variable.

This protocol ensures that the write by thread 1 is committed to main memory before thread 2 attempts to read it, thereby preventing stale or inconsistent values from being observed.

Release or acquire FLUSH

Release and acquire flushes enforce consistency between the memory views of two synchronizing threads, rather than between a thread and main memory directly.

A **release flush** guarantees that any prior operation that writes or reads a shared variable will appear to complete before any operation that writes or reads the same shared variable and follows an acquire flush with which the release flush synchronizes. Concretely, a release flush propagates the values of all shared variables in the thread's temporary view to memory prior to performing any subsequent atomic operation that may establish a synchronization.

An **acquire flush** discards any value of a shared variable in the thread's temporary view that has not been written since the last release flush. It then loads any value of a shared variable propagated by a release flush that synchronizes with it into the temporary view, making that value available for subsequent reads.

Release and acquire flushes may also be used to guarantee that a value written to a variable by one thread can be read by a second thread. To accomplish this, the programmer must ensure that the second thread has not written to the variable since its last acquire flush, and that the following sequence of events occurs in this specific order:

1. The value is written to the variable by thread 1.
2. Thread 1 performs a release flush.
3. Thread 2 performs an acquire flush.
4. The value is read from the variable by thread 2.

This protocol establishes a synchronization edge between the release and acquire operations, ensuring that writes made visible by the release are observable after the acquire.

Example: ***FLUSH example***

Consider the following snippet where multiple threads increment a shared counter:

```
1 #pragma omp parallel
2 {
3     ...
4     #pragma omp atomic update
5     counter++;
6     ...
7     double z = f(counter, ...);
8 }
```

how can you ensure that all the threads have updated `counter` before any thread calls `f`?

Approach 1: The most straightforward solution is to insert a **barrier** before line 8. A barrier implies a strong flush, guaranteeing that all updates are visible to all threads. However, barriers are expensive because they force all threads to synchronize, which is inherently serializing and can severely limit parallel performance.

Approach 2: A more targeted solution uses an **explicit flush** directive to force the thread to read from main memory rather than its local cache:

```
#pragma omp parallel
{
    ...
    #pragma omp atomic update
    counter++;
    ...
    #pragma omp flush(counter) acquire
    double z = f(counter, ...);
    ...
}
```

The `acquire` flush ensures that the thread discards any stale cached value of `counter` and reads the most recent value from memory. This approach is lighter than a full barrier.

Approach 3: For protecting more complex operations involving shared variables, **named critical regions** provide both mutual exclusion and implicit strong flushes:

```
#pragma omp parallel
{
    ...
    #pragma omp critical (update_counter)
    counter++;
    ...
    #pragma omp critical (read_counter)
    double z = f(counter, ...);
    ...
}
```

Named critical regions ensure that only one thread at a time executes each named section, and the implicit flush guarantees memory consistency. Different names allow unrelated critical sections to execute concurrently, reducing contention compared to unnamed critical regions.

Implicit flushes

OpenMP provides implicit flushes at specific synchronization points, eliminating the need for explicit `flush` directives in most cases: but allow recursive acquisition by the owning thread

```
#pragma omp barrier      // Implicit flush of all variables
#pragma omp critical     // Flush on entry and exit
#pragma omp ordered      // Flush on entry and exit
#pragma omp parallel     // Flush on entry (fork) and exit (join)
#pragma omp for          // Flush at implied barrier (if present)
#pragma omp single       // Flush at implied barrier (if present)
```

These implicit flushes apply to *all* shared variables (the so-called "strong flush"), not merely those modified within the region. Additionally, **lock operations** (`omp_set_lock` , `omp_unset_lock`) carry implicit flushes, as discussed in the following section.

Warning: ***No implicit flush for*** `master`

The `#pragma omp master` directive does **not** include an implicit flush. If memory consistency is required after master-only code, an explicit flush or barrier must be inserted.

Memory ordering semantics

OpenMP 5.x adopted the C++11/C11 memory model, exposing fine-grained control over memory ordering. The available orderings, from strongest to weakest, are:

```
seq_cst   // sequential consistency (default)
acq_rel   // acquire-release
release   // release only
acquire   // acquire only
relaxed   // no synchronization guarantees
```

1. `seq_cst` : is the default memory order. It is the most strict memory order and it is the most expensive. A single thread must execute all operations in the order they are written in the code.
2. `acq_rel` : is the acquire-release memory order. It combines acquire semantics on reads and release semantics on writes. It ensures that all memory operations before a release are visible to any thread that performs an acquire on the same synchronization variable. It is less strict than `seq_cst` (no global total order) but sufficient for most producer-consumer patterns.
3. `release` : it is like doing a write back of whatever is in the local cache to the main memory.
4. `acquire` : it is like doing a read from the main memory to the local cache.
5. `relaxed` : is the least strict memory order and it is the cheapest. It is used when you do not need to ensure that all operations are executed in the order they are written in the code.

A `release` operation on a variable ensures that ***all preceding*** memory operations (reads and writes) by the executing thread become visible to any other thread that subsequently performs an `acquire` operation on the same synchronization variable.

```
x = 1;
y = 2;
#pragma omp atomic write release
z = 3; // Release on z synchronizes x, y as well
```

Locks

OpenMP provides explicit lock routines for mutual exclusion when `critical` sections are too coarse-grained or when finer control over synchronization is needed. Unlike named critical regions (which serialize all threads entering a region with the same name), locks can be associated with specific data structures, enabling concurrent access to different parts of a shared resource.

A lock is represented by the `omp_lock_t` type. For scenarios where a thread needs to acquire the same lock multiple times (e.g., recursively), OpenMP provides **nested locks** via `omp_nest_lock_t`. It can be acquired multiple times by the same thread that originally acquired it; each acquisition increments an internal counter, and the lock is only released when this reaches zero.

> ⚠ **Warning**: ***Memory consistency***
>
> Lock operations imply an implicit **flush** of all the thread's visible variables. Specifically, `omp_set_lock` has **acquire** semantics and `omp_unset_lock` has **release** semantics, establishing a happens-before relationship between the lock release and subsequent acquisitions.

Function	**Description**
`omp_init_lock(omp_lock_t *)`	Initialize a lock (must be called before first use).
`omp_destroy_lock(omp_lock_t *)`	Destroy a lock (must not be locked when destroyed).
`omp_set_lock(omp_lock_t *)`	Acquire a lock; **blocks** until the lock is available.
`omp_unset_lock(omp_lock_t *)`	Release a lock (must be called by the owning thread).
`omp_test_lock(omp_lock_t *)`	**Non-blocking** attempt to acquire the lock; returns nonzero on success (lock acquired), zero on failure.

Nested lock variants (`omp_init_nest_lock`, `omp_set_nest_lock`, etc.) follow the same pattern.

Consider a scenario where multiple threads insert elements into a linked list. A global lock serializes all insertions, eliminating parallelism:

```c
omp_lock_t global_lock;
omp_init_lock(&global_lock);

#pragma omp parallel
{
  #pragma omp single nowait
  {
    for (int i = 0; i < N; i++) {
      #pragma omp task firstprivate(i)
      {
        omp_set_lock(&global_lock); // All insertions serialized
        insert_into_list(list, i);
        omp_unset_lock(&global_lock);
      }
    }
  }
}
omp_destroy_lock(&global_lock);
```

A more efficient approach uses **per-node locks**, allowing concurrent insertions at different positions. However, to avoid deadlocks, locks must be acquired in a consistent order.

The function `omp_set_lock` blocks until the lock becomes available, which can lead to deadlock if threads hold locks while waiting for others. The non-blocking `omp_test_lock` returns immediately, allowing the caller to take alternative action if the lock is unavailable:

```
if (omp_test_lock(&lock)) {
    // Lock acquired: perform critical operation
    omp_unset_lock(&lock);
} else {
    // Lock unavailable: retry later or take alternative action
    #pragma omp taskyield  // Yield to allow other tasks to progress
}
```

This pattern is essential for implementing deadlock-free algorithms: if a thread cannot acquire all required locks, it releases those already held and retries, preventing circular wait conditions.

When acquiring multiple locks, always follow these principles:

- **Consistent ordering**: acquire locks in a fixed global order (e.g., by memory address).
- **Try-lock with backoff**: use `omp_test_lock` ; if it fails, release all held locks and retry.
- **Never block while holding**: avoid `omp_set_lock` when already holding another lock

Advanced Concept: ***Lock hints***

OpenMP 5.0 introduced lock hints to guide runtime optimizations:

```
omp_init_lock_with_hint(omp_lock_t *lock, omp_sync_hint_t hint);
omp_init_nest_lock_with_hint(omp_nest_lock_t *lock, omp_sync_hint_t hint);

// Available hints (can be combined with bitwise OR):
omp_sync_hint_none           // No hint (default)
omp_sync_hint_uncontended    // Lock is usually uncontended
omp_sync_hint_contended      // Lock is often contended
omp_sync_hint_nonspeculative // Prefer non-speculative implementation
omp_sync_hint_speculative    // Allow speculative (e.g., HTM-based) implementation
```

These hints allow the runtime to select appropriate implementations: an uncontended lock might use a simple spinlock, while a contended lock might use a queue-based algorithm to reduce cache-line bouncing. Note that as of 2025, compiler support for these hints remains limited; the runtime may ignore unsupported hints.

Tip: ***Memory ordering decisions***

```
Do you need atomicity of an operation?
├─ YES: Use atomic
│     └─ Single variable access?
│           ├─ YES: atomic read/write with acquire/release
│           └─ NO: atomic capture or use lock
└─ NO: Do you need memory ordering?
      ├─ YES: Can you use implicit synchronization (barrier, lock)?
      │     ├─ YES: Use barrier/lock (preferred)
      │     └─ NO: Consider atomic release/acquire pattern
      └─ NO: Use reduction or thread-private variables
```

2.1.6 Controlling the task creation - Clauses

The task creation construct admits a number of clauses that allow to control the creation of tasks:

```
#pragma omp task clause, clause, ...
{
    /* code */
}
```

Clause	Description
`if (expr)`	When `expr` evaluates `false`, the encountering thread does not create a new task in the task queue; instead, the execution of the current task is suspended and the newly created task is *undeferred* and started. The task suspended is resumed afterwards.
`final (expr)`	The `final(expr)` clause can be used to suppress the task creation and then to control the tasking overhead, especially in recursive task creation. If `expr` evaluates `true`, no more tasks are generated and the code is executed immediately. That is propagated to all the children tasks. That is called an *included* task and is always undeferred. Use `omp_in_final()` to check if the task is a final one.
`mergeable`	This clause avoids a separated data environment to be created if a task is undeferred or included.
`tied, untied`	This clause lets a suspended task be resumed by a different thread than the one that started it. The default option is `tied`.
`depend, priority`	*We will cover this afterwards*

Task creation in loops

Many times happens that you need to create tasks in a loop (for instance, a task for every entry, or sections, of an array). The taskloop construct has been conceived to ease this cases, combining the for loops and the tasks natively.

```
#pragma omp taskloop [clause [[,] clause] ...]
{
    for-loops // (perfectly nested)
}
```

Clauses are very similar to both the usual for and task constructs: `private`, `firstprivate`, `lastprivate`, `shared`, `default`, `if`, `final`, `priority`, `untied`, `mergeable`

There are 3 specific clauses for `taskloop`:

- `grainsize(arg)`: `arg` is a positive integer. This clause regulates the granularity of the work assignment, ensuring that the amount of work per task is not too small. The number of loop iterations assigned to each task is at least the grainsize and at most $2\times$ grainsize, but not more than the number of iterations left.
- `num_tasks(arg)`: `arg` is a positive integer, indicating the maximum number of tasks that can be generated at runtime. This is useful to limit the task creation overhead.
- `nogroup`: Specifies that the tasking construct is not embedded in an otherwise implied `taskgroup` construct.

2.1.7 Task priority and dependencies

Task priority

Even when tasks can execute concurrently, it is sometimes beneficial for certain tasks to run earlier than others. For instance, tasks that receive data should typically execute before tasks that post-process the results. The `priority(p)` clause allows the programmer to suggest a scheduling preference to the OpenMP runtime: higher values of `p` indicate higher priority.

```
#pragma omp parallel
#pragma omp single
{
    #pragma omp task priority(100)
    read_data(...);
    #pragma omp task priority(50)
    process_and_save_data(...);
    #pragma omp task priority(10)
    postprocess_and_send_data(...);
}
```

Note that priority is a *hint* to the scheduler, not a strict ordering guarantee. The runtime may ignore priorities under certain conditions (e.g., when work-stealing from another thread's queue).

Data races and dependencies

A ***data race*** leads to undefined behavior; it occurs when three conditions are simultaneously met:

i two or more threads access the same memory location,
ii at least one access is a write
iii the accesses are not ordered by synchronization.

> **Tip**: ***Debugging data races***
>
> The compiler flag `-fsanitize=thread` enables runtime detection of data races:
>
> `gcc -fopenmp -fsanitize=thread -g -o program program.c`

In task-based parallelism, data dependencies frequently arise: one task may need the results produced by another, or must wait for another task's operations to complete before proceeding. OpenMP provides the `depend` clause to express these relationships explicitly, allowing the runtime to schedule tasks in a valid order while maximizing parallelism.

OpenMP recognizes three primary **dependence types**:

- `in`: the task depends on any *previously generated* task that has an `out`, `inout`, or `mutexinoutset` dependence on the same memory region.
- `out` (and `inout`): the task depends on any previously generated task with an `in`, `out`, `inout`, or `mutexinoutset` dependence on the same memory region.
- `mutexinoutset`: similar to `out`, but tasks with `mutexinoutset` on the same variable are *mutually exclusive*: they may execute in any order, but never concurrently.

> **Observation**: ***`inout` is equivalent to `out`***
>
> In OpenMP 5.x, `inout` is treated as an alias for `out`. The distinction is purely semantic (indicating that the task both reads and writes a variable); the scheduling behavior is identical.

The dependence types above correspond to well-known data hazards from instruction parallelism:

- **Read-after-Write (RaW)**
  also called a *true dependency* or *flow dependency*, occurs when a task must read data produced by a previous task. This is the fundamental producer-consumer relationship:

```
#pragma omp task depend(out:result)
compute_result(&result);

#pragma omp task depend(in:result)
use_result(&result);
```

- **Write-after-Read (WaR)**
  also known as an *anti-dependency*, occurs when a task overwrites data that a previous task is still reading. The write must wait until all prior reads complete:

```
#pragma omp task depend(in:buffer)
read_buffer(&buffer);

#pragma omp task depend(out:buffer)
overwrite_buffer(&buffer);
```

- **Write-after-Write (WaW)**
  also known as *output dependency*, occurs when two tasks write to the same location. The final value must come from the logically later task. This is sometimes called a *false dependency* or *name dependency*, since renaming the variable could eliminate it:

```
#pragma omp task depend(out:x)
initialize_x(&x);

#pragma omp task depend(out:x)
reinitialize_x(&x);
```

- **Read-after-Read (RaR)**
  involves two tasks reading the same memory region. Since neither modifies the data, there is no actual dependency; both tasks can execute concurrently:

```
#pragma omp task depend(in:data)
analyze_data(&data);

#pragma omp task depend(in:data)
visualize_data(&data);
```

Tip: ***OpenMP 5.x synchronization best practices***

When choosing synchronization mechanisms, prefer higher-level constructs that are easier to reason about:

1. **First choice**: High-level constructs (`barrier` , `critical` , `atomic`)
2. **Second choice**: Atomics with explicit memory order (`acquire` / `release`)
3. **Last resort**: Explicit `flush` (only when synchronizing multiple variables at once)
4. **Avoid**: `volatile` provides no synchronization guarantees in OpenMP

3 Supercomputers

Modern scientific discovery increasingly relies on high-performance computing infrastructure capable of handling massive computational workloads. Supercomputers represent the pinnacle of this infrastructure, combining thousands of processors, accelerators, and specialized networking hardware to tackle problems that would be intractable on conventional systems. These range from climate modeling and molecular dynamics simulations to machine learning training and quantum mechanics calculations. We will focus on Leonardo, one of Europe's flagship supercomputing systems. We'll examine its modular design, computing capabilities, and the practical aspects of accessing and utilizing it.

3.1 Leonardo Supercomputer

Leonardo is a pre-exascale supercomputer hosted at CINECA in Bologna, Italy, and represents one of the cornerstones of the European High Performance Computing Joint Undertaking (EuroHPC JU). Leonardo was designed to support both traditional HPC workloads and emerging AI applications.

The system exemplifies the trend toward heterogeneous computing in modern HPC, integrating both CPU-centric compute nodes and GPU-accelerated nodes optimized for highly parallel workloads.

3.1.1 System Architecture and Modules

Leonardo adopts a modular architecture consisting of several distinct partitions, each tailored to specific classes of workloads. The two primary modules are the **Booster module** and the **Data-Centric (DHPC) module**, supplemented by additional service and hybrid partitions.

Booster Module

The Booster module constitutes the heart of Leonardo's computational power and is optimized for massively parallel, compute-intensive workloads. This partition is composed of GPU-accelerated nodes, each equipped with:

- **CPUs**: 1 Intel Xeon Platinum 8358 processor (32 cores, 2.6 GHz base frequency)
- **GPUs**: 4 NVIDIA A100 GPUs (64 GB HBM2 memory each), providing substantial computational throughput for floating-point operations and tensor computations
- **System Memory**: 512 GB DDR4 RAM per node
- **Node-to-GPU Interconnect**: GPUs connected via NVIDIA NVLink for high-bandwidth, low-latency communication within each node

The Booster module comprises approximately 3,500 such nodes, yielding roughly 14,000 NVIDIA A100 GPUs in total. This configuration is particularly well-suited for applications in machine learning, computational fluid dynamics, materials science, and other domains that benefit from the massive parallelism offered by modern GPUs. Each A100 GPU provides mixed-precision capabilities (FP64, FP32, FP16, and Tensor Core operations), enabling efficient execution of both traditional HPC simulations and AI training workloads.

Data-Centric (DHPC) Module

The Data-Centric module provides CPU-centric compute resources designed for workloads that require large memory capacity, complex branching logic, or I/O-intensive operations. Nodes in this partition feature:

- **CPUs**: 2 Intel Xeon Platinum 8358 processors per node (64 cores total)
- **System Memory**: Typically 512 GB DDR4 RAM per node, with some high-memory configurations available
- **Local Storage**: NVMe SSDs for fast local data staging

This module comprises approximately 1,500 nodes and is intended for traditional HPC applications such as computational chemistry codes, finite element analysis, and large-scale data analytics tasks that do not map efficiently onto GPU architectures.

Inter-Node Network Topology

Leonardo employs a **Dragonfly+** network topology based on NVIDIA Mellanox InfiniBand HDR (High Data Rate) technology and NVIDIA Quantum QM8700 switches. This topology is designed to provide high bandwidth and low latency while maintaining scalability and resilience.

In the Dragonfly+ topology, compute nodes are organized into **groups** (or cells), with dense all-to-all connectivity within each group. Inter-group connections are carefully engineered to minimize the maximum number of hops between any two nodes while balancing cost and performance. Each node is connected via InfiniBand HDR links operating at 200 Gb/s, ensuring that communication-intensive parallel applications can scale efficiently across thousands of nodes.

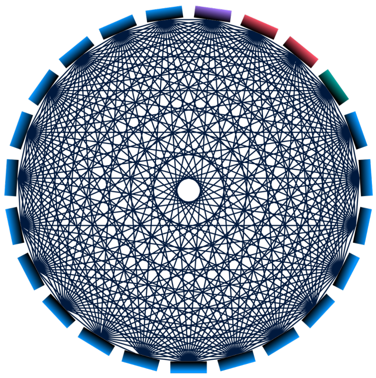


Figure 3.1: Dragonfly+ Topology. Booster Module nodes (blue), I/O cell (purple), Data-centric cells (red), and Hybrid cell (green) [**cinecaLeonardoSystem**].

3.1.2 Storage System

Leonardo's storage infrastructure is organized into a two-tier hierarchy:

- **Fast Tier (NVMe-based)**: 5.4 PB of capacity with an aggregate bandwidth of 1.4 TB/s. This tier uses high-speed NVMe SSDs and is intended for I/O-intensive workloads requiring rapid data access, such as checkpointing in large-scale simulations or intermediate data staging for machine learning training.
- **Capacity Tier (HDD-based)**: 106 PB of capacity with an aggregate bandwidth of 2.9 TB/s. This tier provides long-term storage for datasets, simulation outputs, and archival purposes. While slower than the fast tier, it offers significantly greater capacity at a lower cost per terabyte.

Both tiers are accessible via a parallel filesystem (typically GPFS/Spectrum Scale or Lustre) that provides a unified namespace and supports concurrent access from thousands of compute nodes.

Observation: *Top500*

The **Top500** list is the authoritative ranking of the world's fastest supercomputers, updated twice a year (June and November). Leonardo currently holds the rank of 9th fastest supercomputer globally. At its debut in November 2022, Leonardo achieved the impressive position of **4th fastest**, trailing only Frontier (USA), Fugaku (Japan), and LUMI (Finland).

3.1.3 Accessing Leonardo

Access to Leonardo is managed through a project-based allocation system. For users affiliated with Italian universities and research institutions, CINECA offers several account classes tailored to different project sizes and durations:

Tip: ***Apply for resources***

If you belong to an Italian university, you can apply for resources in the following classes:

- **Class B**: Large-scale projects, typically requesting millions of core-hours. Duration: 12 months. Calls for proposals are issued twice per year.
- **Class C**: Small to medium-sized projects. Duration: 9 months. Continuous submission process, with up to 10 calls per year.
- **Class D**: Storage allocations related to HPC simulations, useful for projects requiring substantial long-term data retention. Duration: 36 months.
- **Test Account**: Small allocations for testing, debugging, and scaling studies. Duration: 3 months. Ideal for validating code performance before requesting larger allocations.

Upon logging in to Leonardo, users are greeted by the **Message of the Day** (MOTD), an informational banner designed to keep users up-to-date with the system's status and important announcements. The MOTD typically includes:

- A brief description of the cluster architecture or key features
- Real-time system status updates (e.g., queues, ongoing issues, resource availability)
- "In evidence" messages highlighting noteworthy news or actionable items
- Important notifications, such as planned downtime, upcoming maintenance, or urgent advisories

3.2 Working on Leonardo

3.2.1 Filesystem Organization

Upon logging into Leonardo, users find themselves in a hierarchical filesystem with several designated directories, each serving a specific purpose:

- **HOME** (`$HOME`): A personal home directory; it is backed up and is intended for configuration files, scripts, and small datasets. Quota-limited, typically to tens of gigabytes.
- **WORK** (`$WORK`): A larger, project-specific workspace for active development, source code, and intermediate build artifacts. Not typically backed up, but persistent across sessions.
- **SCRATCH** (`$SCRATCH`): A high-performance scratch space for temporary files generated during job execution. This area offers the best I/O performance and is intended for files needed only during the runtime of a job. Files in scratch are subject to automatic purging policies (e.g., files not accessed for 40 days may be deleted), so do not use this for long-term storage.
- **DATA** (`$DATA`): Intended for long-term storage of large datasets and simulation outputs. This directory is often mapped to the capacity storage tier and may have different quota and performance characteristics than `$SCRATCH`.

- **PUBLIC**: A shared directory where project members can exchange data and scripts. Useful for collaboration within a project team.

Active simulation I/O should target `$SCRATCH` for speed, while final results should be moved to `$DATA` or `$WORK` for safekeeping.

3.2.2 Software and Module Environment

Leonardo's software stack is managed via the **Environment Modules** system, with packages built and deployed using the **Spack** package manager. This approach allows for coexistence of multiple versions and configurations of libraries and compilers, enabling reproducibility and flexibility.

Upon login, a default environment is loaded, but users typically need to load additional modules to access specific compilers, MPI implementations, or scientific libraries.

The module system provides commands to query, load, and manage the software environment:

- `module avail` or `module av` : List all available modules.
- `module list` : Show currently loaded modules and profiles.
- `module load <name>` : Load a specific module into the environment.
- `module unload <name>` : Unload a module.
- `module purge` : Remove all loaded modules (useful for starting with a clean environment).
- `module show <name>` : Display information about what a module does (env. variables, paths).
- `modmap -m <name>` : Search or map module names (CINECA-specific tool).

Different *profiles* or module collections may be available depending on whether you are working on CPU-only or GPU-enabled nodes, or whether you are using a specific programming model (MPI, CUDA, OpenMP, OpenACC, etc.).

Programming Environments

Leonardo supports multiple compiler toolchains and programming environments:

- **GNU Compiler Collection (GCC)**: Open-source compilers (`gcc` , `g++` , `gfortran`).
- **NVIDIA HPC SDK**: Provides `nvc` , `nvc++` , `nvcc` , `nvfortran` with OpenACC and CUDA support, essential for GPU programming on the Booster module.
- **Intel oneAPI**: Compilers (`icc` , `icpc` , `ifort`) optimized for Intel CPUs, along with Intel MPI.

To develop GPU-accelerated code, load the NVIDIA HPC SDK module (e.g. `module load nvhpc`) and ensure that the CUDA runtime libraries are accessible. For traditional MPI applications, load an MPI module compatible with the chosen compiler (e.g. `module load openmpi`).

⚠ **Warning**: ***Intel and NVIDIA Compilers***

Important: Intel compilers do *not* support CUDA or GPU offloading on NVIDIA GPUs. If your code targets NVIDIA GPUs using CUDA, OpenACC, or similar technologies, you must use the NVIDIA HPC SDK tools for compilation.

3.2.3 Job Submission with Slurm

CINECA HPC clusters, such as Leonardo, are shared among many users. As such, responsible usage is essential to ensure fair access and optimal performance for everyone. The system is divided into two main types of nodes: login nodes and compute nodes.

When you log in, you access a **login node**, intended only for light tasks such as editing files, compiling code, or performing brief test runs. Running heavy computations or large parallel jobs directly on it is strictly prohibited. The login environment enforces a hard CPU time limit for any process, and jobs exceeding this limit may be killed without warning. Furthermore, GPUs are not available on login nodes, so you cannot test or run GPU code interactively in this environment.

All resource-intensive and production workloads must be submitted to the **compute nodes** through the ***Slurm*** job scheduler. Slurm manages how resources like CPU cores, GPUs, memory, and temporary storage are allocated to different users. Compute nodes support two main job submission styles: batch mode, in which you provide a job script for Slurm to execute, and interactive mode, where you request an interactive session on a compute node for active development or debugging.

While compute nodes themselves are shared across users, once Slurm assigns resources to a job, those resources are dedicated exclusively to that job for its duration. It is crucial to request only the resources you truly need and to release them promptly to maximize efficiency and minimize costs, since accounting is based on *reserved* resources rather than those you actively use.

Batch Job Submission

A typical batch script for Slurm on Leonardo includes a shebang line, a series of `#SBATCH` directives specifying resource requirements, environment setup commands (loading modules), and finally the execution command. Below is an example for a GPU job:

```bash
#!/bin/bash

# Slurm directives
#SBATCH --job-name=my_gpu_job       # Job name
#SBATCH --nodes=1                   # Number of nodes
#SBATCH --ntasks-per-node=1         # Number of MPI tasks per node
#SBATCH --cpus-per-task=8           # Number of CPU cores per task
#SBATCH --gres=gpu:4                # Request 4 GPUs per node
#SBATCH --mem=494000                # Memory in MB (approx 480 GB)
#SBATCH --time=00:30:00             # Wall-clock time limit (hh:mm:ss)
#SBATCH --partition=boost_usr_prod  # Partition (queue) to submit to
#SBATCH --account=tra25_gputs       # Account/project code
#SBATCH --reservation=s_tra_gputs   # (Optional) reservation name
#SBATCH --output=job_%j.out         # Standard output file (%j = job ID)
#SBATCH --error=job_%j.err          # Standard error file

# Load necessary modules
module <module-name>

# Run the executable
srun my_application # or mpirun
```

Key directives:

- `--nodes` and `--ntasks-per-node`: Define the number of nodes and MPI ranks per node.
- `--cpus-per-task`: Sets CPU cores per MPI task (useful for hybrid MPI+OpenMP jobs).
- `--gres=gpu:N`: Request `N` GPUs per node. On Booster nodes, typically `gpu:4`.
- `--time`: Maximum wall-clock time. Jobs exceeding this limit are terminated.
- `--partition`: Specifies the queue or partition. Common partitions on Leonardo include

`boost_usr_prod` (Booster nodes) and `dcgp_usr_prod` (DHPC nodes).
- `--account`: Your project or account code, used for tracking resource usage and billing.

To submit the job, save the script (e.g., `job.sh`) and run:

```
sbatch job.sh
```

Slurm will return a job ID, which you can use to track the job's status.

Interactive Jobs

For debugging, performance profiling, or exploratory work, you may request an interactive session on compute nodes. Use `salloc` or `srun` with the `--pty` option:

- `srun`:

  ```
  srun -N=1 --ntasks-per-node=1 --cpus-per-task=8 --gres=gpu:1 --time
      =00:10:00 --partition=boost_usr_prod --account=<name> --pty bash
  ```

  Once the allocation is granted, you will receive an interactive shell on a ***compute node***, which means the resources are allocated and the shell is executed directly on the worker node.
- `salloc`:

  ```
  salloc -N=1 --ntasks-per-node=1 --cpus-per-task=8 --gres=gpu:1 --time
      =00:10:00 --partition=boost_usr_prod --account=<name>
  ```

  A new session starts on the ***login node***, as `salloc` only performs the resource allocation. Therefore, you must explicitly launch commands (usually via `srun`) to execute code on the allocated compute node(s). Useful to run multiple commands within the same allocation.

> ⚠ **Warning**: ***Exit the allocation***
>
> Remember to exit or cancel the allocation when done to avoid wasting resources.

Monitoring and Managing Jobs

When specifying the `--account` option, you must use your project account. Each account has a budget of core-hours shared among team members. On Leonardo, you can check the balance with:

```
saldo -b          # For Booster accounts
saldo -b --dcgp   # For DCGP accounts
```

Booster and DCGP accounts/resources are separate, and can only be used on their respective partitions. Each **Booster node** provides up to 32 CPU cores, 4 GPUs, and approximately 494 GB RAM. You can allocate resources for your job within these limits, but the product `ntasks-per-node` × `cpus-per-task` must not exceed 32. Accounting for allocation will consider the number of CPUs, GPUs, and RAM you request.

To monitor and control your jobs, you can use the following SLURM commands:

- `squeue -u <username>` or `squeue --me`: Shows the list of all your scheduled jobs, along with their status (pending, running, closing, …). Also displays the jobID.

- `scontrol show job <jobid>`: Provides a long list of information for the job requested, including resource allocation, start time, and node assignments. If the job is not running yet, you'll be notified about the reason and, for top priority jobs, an *estimated start time* is provided.
- `scancel <jobid>`: Removes (cancels) the job (queued or running) from the scheduled job list.
- `sacct <options> <jobid>` (e.g., `sacct -Bj <jobid>`): Displays accounting data for all jobs and job steps in the SLURM log or database. Useful for post-mortem analysis of resource usage.
- `sinfo` (e.g., `sinfo -o "%10D %a %20F %P"`): Provides information about SLURM nodes and partitions.

4
CUDA C/C++

4.1 Introduction to Accelerated Computing

The constant demand for computational power in science and engineering has driven the evolution of computer architecture. For decades, performance gains were fueled by increasing clock frequencies and instruction-level parallelism, but physical limitations have shifted the focus toward massive parallelism. This paradigm shift requires a corresponding evolution in programming models to effectively harness the capabilities of modern hardware.

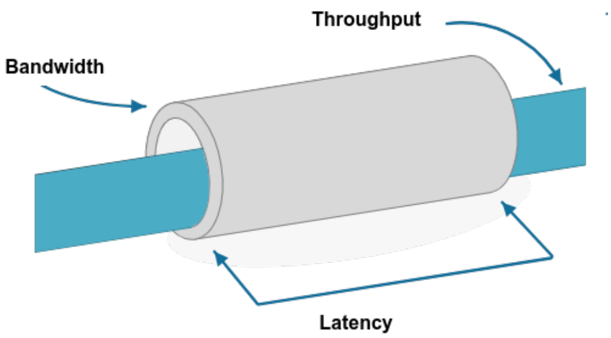
This chapter introduces *accelerated computing*, a paradigm where a specialized, highly parallel processing unit, the Graphics Processing Unit (GPU), is used in conjunction with a traditional Central Processing Unit (CPU) to accelerate computationally intensive portions of an application. We will explore the architectural differences between CPUs and GPUs, delve into the CUDA C/C++ programming model for NVIDIA GPUs, and discuss fundamental concepts such as memory hierarchies, thread organization, and performance optimization.

Key Performance Metrics

When evaluating system performance, three metrics are crucial:

- **Latency**: The time it takes to perform a single operation or access a single piece of data (e.g., time to fetch a value from memory). Measured in seconds.
- **Bandwidth**: The rate at which data can be transferred. Measured in bytes per second.
- **Throughput**: The total number of operations completed in a given amount of time. Measured Floating-Point Operations Per Second (FLOPS).

CPUs are optimized for low latency, while GPUs are designed for high throughput and high memory bandwidth, sacrificing single-thread latency to achieve massive parallelism.



⚠ Warning: ***Architecture and Programming Models***

A fundamental principle in HPC is that changes in hardware architecture necessitate changes in programming models to achieve efficiency. Code written for a latency-oriented CPU will not perform well on a throughput-oriented GPU.

4.1.1 Architectural Design Philosophies: Latency vs. Throughput

CPUs and GPUs are designed with fundamentally different goals, their architectures reflect this.

Latency-Oriented Design (CPUs)

A CPU is a **latency-oriented** device, designed to execute a single thread of instructions as quickly as possible:

- **Powerful Cores**: A CPU has a small number of powerful, general-purpose cores capable of complex control flow and integer/floating-point arithmetic.
- **Large Caches**: It employs a deep, multi-level cache hierarchy (L1, L2, L3) to minimize memory latency. The goal is to keep data as close to the core as possible, to reduce waiting time for data from main memory.
- **Sophisticated Control Logic**: Features as branch prediction, speculative execution are used to maximize performance.

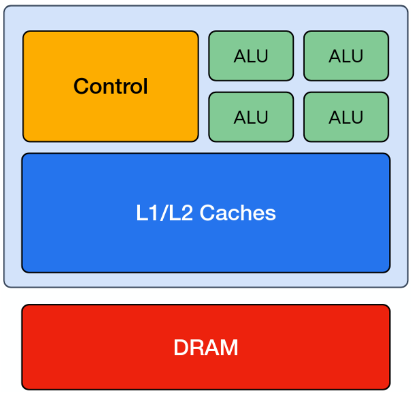


Figure 4.1: Simplified CPU [**cinecaSCAITraining**]

This design excels at serial, irregular workloads where instructions may depend on previous ones.

Modern architectures rely on a memory hierarchy to balance the trade-off between fast, small, and expensive memory (CPU caches) and large, slower, but cheaper memory (DRAM or storage).

Figure 4.2: Memory Hierarchy [**cinecaSCAITraining**]

Virtual Memory: The Great Abstraction

Virtual memory is a foundational abstraction in all modern CPUs, allowing each process to believe it has access to a private, contiguous range of addresses, providing several advantages:

- **Simplified Programming Model**: Programs can use memory addresses freely.
- **Isolation**: Each process is protected from accidental or malicious access to others' memory.
- **Efficient Resource Utilization**: The OS can move data between physical memory and storage transparently.

The virtual address used by a program is translated into a physical address by the CPU hardware (the Memory Management Unit, MMU) using page tables maintained by the operating system.

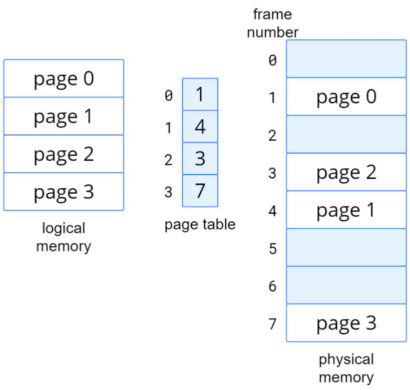


Figure 4.3: Translation of virtual to physical addresses.

> **⚠ Warning:**
>
> Page Faults and Swapping If a program accesses memory that is not currently mapped to physical RAM, a **page fault** occurs. The OS may need to load the required page from disk (swap space), causing a major slowdown.

When a CPU needs data, it first checks the fastest cache (L1), then proceeds to L2, L3, and finally DRAM if necessary. The principle of **locality** (recently or nearby accessed data being more likely to be reused) underpins cache design.

Caching increases effective memory bandwidth and reduces average memory access latency—key for maximizing CPU (and to a certain extent, GPU) performance.

MISSING: multicore CPUs

Throughput-Oriented Design (GPUs)

A GPU, in contrast, is a **throughput-oriented** device, designed to execute thousands of parallel threads simultaneously. Its architecture is specialized for data-parallel tasks:

- **Many Simple Cores**: A GPU contains thousands of simpler, more energy-efficient cores grouped into *Streaming Multiprocessors* (SMs).
- **Smaller Caches, High-Bandwidth Memory**: While GPUs have caches, they are smaller relative to the number of cores. The design prioritizes high memory bandwidth to feed the massive number of cores with data concurrently. The philosophy is to ”hide” latency by having other work to do. While one group of threads waits for data, the SM switches to another group that is ready to execute.
- **Simple Control Logic**: GPUs use simpler control logic and are optimized for executing the same instruction on multiple data elements (a model known as SIMT, or Single Instruction, Multiple Threads).

This design is ideal for workloads where the same computation can be performed independently on many different data points, such as matrix multiplication, image processing, and scientific simulations.

4.2 The GPU Architecture

In a typical accelerated node, the CPU is referred to as the **host** and the GPU as the **device**. They are physically distinct processors with their own memory spaces, connected by a high-speed interconnect like the PCIe bus. A key task in accelerated computing is managing the transfer of data and control between the host and device.

GPUs are designed to execute a massive number of threads simultaneously. In the CUDA model, threads are launched in groups of 32, known as **warps**. All threads in a warp execute the same instruction at the same time, making them the fundamental unit of scheduling on the GPU. This execution model is highly efficient for data-parallel problems but requires careful programming to avoid performance pitfalls.

4.2.1 NVIDIA Ampere Architecture

The NVIDIA A100 GPU, used in the Leonardo supercomputer, is based on the **Ampere architecture**. A full Ampere GPU consists of several **Graphics Processing Clusters (GPCs)**, which are further subdivided into **Streaming Multiprocessors (SMs)**. The SM is the core processing unit of the GPU.

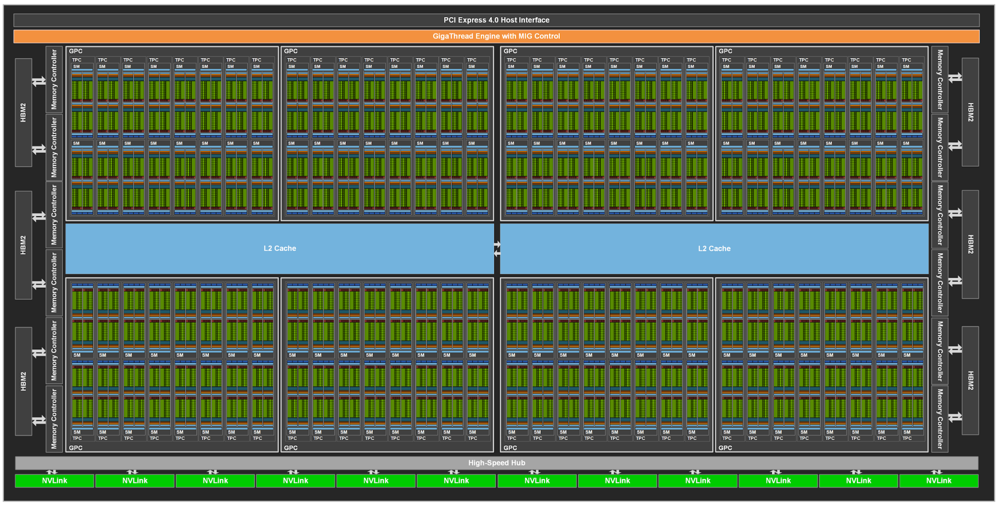


Figure 4.4: High-level overview of the NVIDIA Ampere A100 GPU architecture, showing the arrangement of GPCs and SMs.

The **Streaming Multiprocessor (SM)** is the heart of the NVIDIA GPU. In the Ampere architecture, each SM is partitioned into four processing blocks, each with its own instruction buffer, warp scheduler, and dispatch units. This allows the SM to manage and execute multiple warps concurrently.

Key components within an Ampere SM include:

- **FP32/FP64 Cores**: CUDA cores dedicated to single- and double-precision floating-point arithmetic.
- **Tensor Cores (3rd Gen)**: Specialized units that accelerate mixed-precision matrix multiply-accumulate operations, crucial for AI and deep learning workloads. They support new data formats like TF32 and BFloat16.
- **Shared Memory and L1 Cache**: A configurable block of high-speed on-chip memory that can be used as a software-managed cache (`L1`) or a scratchpad for thread cooperation (`shared memory`).
- **Warp Schedulers**: Hardware units that select which warps are ready to execute their next instruction.

The SM is designed to keep its execution units busy by rapidly context-switching between different warps. If one warp stalls (e.g., waiting for data from global memory), the scheduler immediately switches to another resident warp that is ready to execute, thereby hiding memory latency and maximizing computational throughput.

Figure 4.5: Simplified block diagram of an NVIDIA Ampere Streaming Multiprocessor (SM).

4.3 CUDA Programming Model

`<<< >>>`: are used to quantify the number blocks and threads to launch the kernel.

We have dirrerent qualifiers for the functions:

- `__global__`: It indicates that the function will run in the GPU, and can be invoked generally both from the host and the device.

```
__global__ void GPUFunction() {
    printf("Hello from GPU!\n");
}

int main() {
    CPUFunction();
    GPUFunction<<<1, 1>>>();
    cudaDeviceSynchronize();
}
```

- `__device__`: device function, executed on the device.
- `__host__`: host function, executed on the host.
- `__host__ __device__`: host and device function, compiled for both.

A cuda kernel is executed as a grid (array) of threads, each of them runs in the same kernel.

Each thread has a unique index `threadIdx.x`, which is used to access the data.

To compile a program we must use the NVIDIA `nvcc` compiler. It is able to identify the functions to be executed on the GPU and the ones to be executed on the host.

```
nvcc -arch=sm_80 -o <output-file> <cuda-code>.cu -run
```

Where:

- `-arch=sm_80`: specifies the architecture of the GPU to compile for.
- `-run`: runs the program after compilation.

SIMT vs SIMD

In cuda programming, we use the SIMT (Single Instruction Multiple Thread) model.

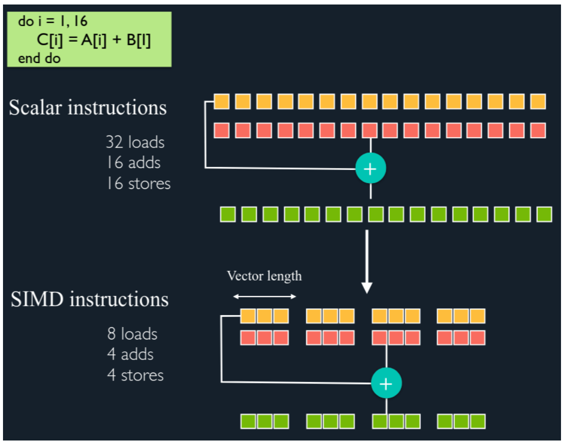


Figure 4.6: SIMT vs SIMD

- **SIMD**: “One instruction, one data chunk.”
  - A single instruction operates on multiple data elements simultaneously
  - Requires wide vector units in hardware (e.g., 4 or 8 lanes)
  - A SIMD register (or a vector register) can hold many values (2 - 16 values or more) of a single type
  - Operates on packed data in wide registers (e.g., 128-bit or 256-bit)
  - All elements in the vector must follow the same control flow—no divergence
  - Vectorisation helps you write code which has good access patterns to maximise bandwidth
- **SIMT** ”Single Instruction, Multiple Threads”.
  - Uses scalar execution units, not wide vectors, rigid, lockstep vector processing
  - 32 threads in a warp share a single instruction fetch, executed over multiple cycles (e.g., 4 cycles on 8 CUDA cores)
  - Flexible, thread-level parallelism with divergence support.
  - Single instruction, multiple flow paths if statements are allowed!

SIMT allows CUDA GPU to perform “vector” computations on scalar cores. Much easier to vectorise than getting compiler to autovectorize on CPU.

4.4 GPU Thread Hierarchy

A gpu consists of thousands of grids, each one containing thousands of thread blocks (teams), each one containing 1024 threads divided into warps of 32 threads each.

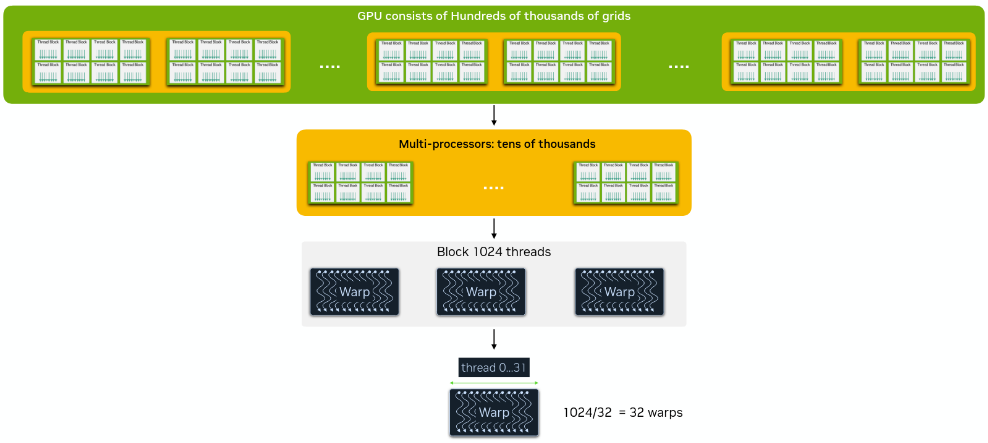


Figure 4.7: GPU Thread Hierarchy

All thread blocks in a grid must have the same number of threads, ensuring homogeneity across the GPU workload. To process N elements in parallel, we need to launch at least N concurrent threads on the device. Threads within the same block (or team) can efficiently cooperate by exchanging data through a shared memory cache. Importantly, each block is executed independently, and there is no guarantee on the order in which blocks are scheduled or executed by the GPU.

CPUs and GPUs have physically distinct memory regions connected by the PCIe bus.

Every time we want to compute something on a GPU we need to perform some steps:

- Allocate GPU memory
- Copy data from the host to the GPU

- Load GPU program and execute, caching data on chip for performance
- Copy results from the GPU memory to the host memory
- Free the GPU memory

We can see how the code for cpu is very similar to the one for gpu, but with the addition of the cuda functions.

CPU code

```
int N = 10000;
size_t size = N * sizeof(int);

int *a;
a = (int *) malloc(size);

free(a);
```

GPU code

```
int N = 10000;
size_t size = N * sizeof(int);

int *a;
cudaMallocManaged(&a, size);

cudaFree(a);
```

Choosing the optimal grid size

Let's start by choosing the ***optimal block size***. To do that, we need to write an execution configuration that creates more threads than necessary, then pass a value as an argument into the kernel (N) that represents that total size if the data set to be processed/total threads needed to complete the work.

Then, we need to calculate the global index and if it does not exceed N perform the kernel work.

```
__global__ vectorSum(int N) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    if (idx < N) { // only do work if it does}
}
```

Maximum size at each level of the thread hierarchy is device dependent.

On A100 typicaly you get:

- Maximum number of threads per block: 1024
- Maximum sizes of x-, y-, and -z dimensions of threads block: 1024 x 1024 x 64
- Maximum sizes of each dimension of grid of thread blocks: 65535 x 65535 x 65535 (about 280,000 billion blocks)

The best performance is achieved for blocks that contain a number of threads that is a multiple of 32, due to GPU hardware traits.

Example: ***Choosing the optimal block size***

We want to run 1000 parallel task with blocks containing 256 threads. How do we choose the optimal block size?

```
int N = 100000; size_t threads_per_blocks = 256;
size_t number_of_blocks = (N + threads_per_block - 1) /
    threads_per_block;
kernel<<<number_of_blocks, threads_per_block>>>(N);
```

The ”-1” term is added to round up the division if necessary.

A limited number of threads (1024) can fit inside a thread block To increase parallelism, we need to coordinate work among thread blocks

This is achieved by mapping element of data vector to threads using global index

```
int idx = threadIdx.x + blockIdx.x * blockDim.x;
```

Error handling

All CUDA APIs returns an error code of type `cudaError_t`. A special value `cudaSuccess` is returned on success, otherwise an error code is returned.

To understand wich problem occurred, we can use the cudaGetErrorString() function.

```
    cudaError_t err;
    err = cudaMallocManaged(&a, size);
    if (err != cudaSuccess)
        printf("Error: %s\n", cudaGetErrorString(err));
```

To check for errors occurring at the time of kernel execution, we can use the cudaGetLastError() function.

4.5 NVIDIA profiling tools

A typical scenario while accelerating a code is to start with a cpu-only version and then accelerate it using cuda.

TO understand if everything is working correctly, we start with a **Nsight System profiler**: it is a system-wide performance viewer, which tracks CPU and GPU interactions across time.

If we find something wrong, we can use the **Nsight Compute profiler**: it is a GPU-only profiler, which tracks the occupancy, memory usage, and instruction throughput.

Let's see how to use the Nsight System profiler.

```
nsys profile -t cuda,nvtx,mpi,openacc -stats=true --force-overwrite=true
    -o <output-file> <executable>
```

Nsight Compute profiler

```
nsys profile -t cuda -o <output-file> <executable>
```

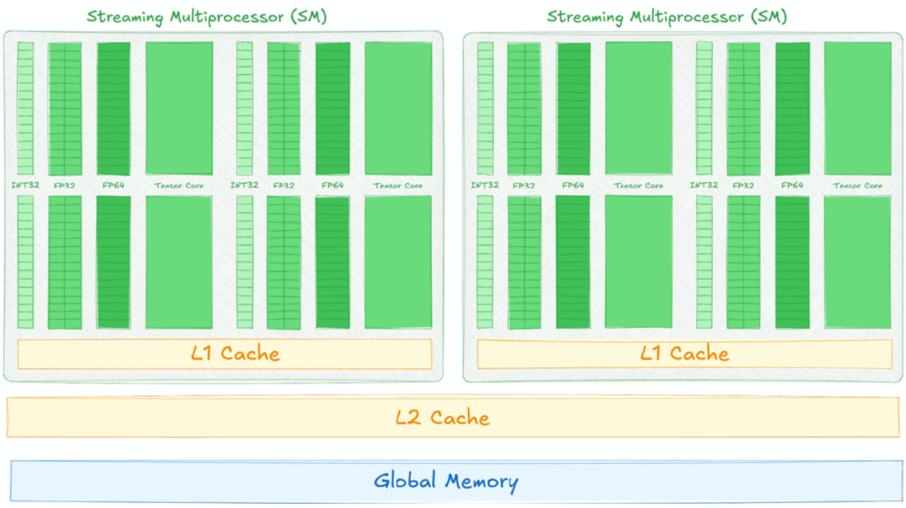


Figure 4.8: Memory Hierarchy

Bandwidth	Latency
13×	1×
3×	5×
1×	15×

Figure 4.9: Bandwidth and Latency comparison

In such architecture, CUDA provides 4 different memory types:

- **Global Memory**: The largest memory space available on the device, but also the slowest in terms of access latency.
  - Scope: Accessible by all threads (from any block or grid).
  - Usage: Memory allocated with `cudaMalloc()` and data transferred using `cudaMemcpy()`.
- **Shared Memory**: Fast and low-latency memory, but much smaller than global memory.
  - Scope: Shared between all threads within the same block.
  - Usage: Useful for data reuse and to reduce global memory accesses. Declared with the `__shared__` qualifier.
- **Registers**: The fastest memory available, mapped directly to the hardware registers of each Streaming Multiprocessor (SM).
  - Scope: Private to each thread; cannot be accessed by other threads.
  - Usage: Used for automatic variables declared within a kernel.
- **Constant Memory**: Read-only memory space cached and optimized for uniform access by all threads. Offers efficient broadcast for all threads reading the same location.
  - Scope: Accessible from all threads, but cannot be modified from device code.
  - Usage: Declared with the `__constant__` qualifier.

Tip: ***Memory coalescing***

For optimal performance, threads in a warp should access consecutive memory addresses. Coalesced memory access can improve bandwidth utilization by up to 10x compared to non-coalesced access patterns.

4.5.1 CUDA Streams

A CUDA stream is a sequence of CUDA operations. The defaualt stream executes those operations in the order they are issued, therefore, an instruction must be completed before the next one can begin.

Multiple streams or Non-default streams can be created and utilise by CUDA programmers. While single stream kernels must execute in order, kernels in different, non-default streams, can interact concurrently, and therefore have no fixed order of execution

This can be useful to overlap the execution of kernels and memory transfers, or to execute kernels in parallel.

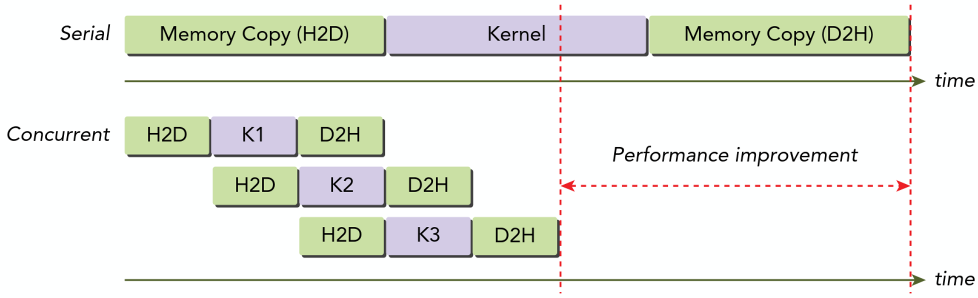


Figure 4.10: CUDA Streams Overlap

```
// Create a new stream
cudaStream_t stream;
cudaStreamCreate(&stream);

// Launch a kernel in the new stream
someKernel<<<numberOfBlocks, numberOfThreads, 0, stream>>>();

// Destroy the stream
cudaStreamDestroy(stream);
```

Advanced Concept: ***non-blocking streams***

cudaStreamCreate creates a blocking stream, but it exitsts also a non-blocking versions, like `cudaStreamNonBlocking` . This topic is not covered in this course.

5
OpenACC

```
#pragma acc <directive> <clauses>
```

`#pragma` in C/C++ is what's known as a "compiler hint." These are very similar to programmer comments, however, the compiler will actually read our pragmas.

Pragmas are a way for the programmer to "guide" the compiler, without running the chance damaging the code. If the compiler does not understand the pragma, it can ignore it, rather than throw a syntax error.

`acc` is an addition to our pragma. It specifies that this is an OpenACC pragma. Any non-OpenACC compiler will ignore this pragma. Even the nvc/nvc++ compiler can be told to ignore them. (which lets us run our parallel code sequentially!)

`directives` are commands in OpenACC that will tell the compiler to do some action. For now, we will only use directives that allow the compiler to parallelize our code.

`clauses` are additions/alterations to our directives. These include (but are not limited to) optimizations. The way that I prefer to think about it: directives describe a general action for our compiler to do (such as, paralellize our code), and clauses allow the programmer to be more specific (such as, how we specifically want the code to be parallelized)

Directives

- `parallel`: a parallel region of code. The compiler generates a parallel kernel for that region. Each gang will execute the entire loop.

  ```
  #pragma acc parallel
  {
      for (int i = 0; i < N; i++)
          A[i] = B[i] + C[i];
  }
  ```

- `loop`: dentifies a loop that should be distributed across threads. Parallel and loop are often placed together

  ```
  #pragma acc parallel
  {
      #pragma acc loop
      for (int i = 0; i < N; i++)
          A[i] = B[i] + C[i];
  }
  ```

- `kernels` a parallel region of code. It allows the programmer to step back, and rely solely on the compiler.

```c
// example of a kernel that performs two different operations
#pragma acc kernels
{
    for (int i = 0; i < N; i++)
        A[i] = B[i] + C[i];

    for (int i = 0; i < N; i++)
        D[i] = A*x[i] + B*y[i];
}

// loop independent: the compiler can parallelize the loop
#pragma acc kernels loop independent
{
    for (int i = 0; i < N; i++)
        A[i] = B[i] + C[i];
}
```

Compiling flags

<table>
<tr><th>Flag</th><th>Description</th></tr>
<tr><td>-acc</td><td>enable OpenACC targeting device</td></tr>
<tr><td>-acc=host</td><td>generate an executable that will run serially on the host CPU</td></tr>
<tr><td>-acc=multicore</td><td>parallelize for a multicore CPU</td></tr>
<tr><td>-acc=gpu -gpu=cc80</td><td>map OpenACC parallelism to an NVIDIA GPU, compile targ</td></tr>
<tr><td>-gpu=managed</td><td>place all allocatables in CUDA Unified Memory</td></tr>
<tr><td>-gpu=pinned</td><td>use CUDA pinned memory for all allocatables</td></tr>
<tr><td>-Minfo=accall</td><td>compiler diagnostics for OpenACC</td></tr>
<tr><td>export NVCOMPILER_ACC_NOTIFY=1|2|3</td><td>environment variable for NOTIFY</td></tr>
<tr><td>-mp</td><td>enable OpenMP targeting device</td></tr>
<tr><td>-mp=gpu</td><td>to generate an executable that will run serially on the host CPU</td></tr>
<tr><td>-gpu=cc80</td><td>map OpenACC parallelism to an NVIDIA GPU, compile targ</td></tr>
<tr><td>-gpu=managed</td><td>place all allocatables in CUDA Unified Memory</td></tr>
<tr><td>-gpu=pinned</td><td>use CUDA pinned memory for all allocatables</td></tr>
<tr><td>-Minfo=accall</td><td>Compiler diagnostics for OpenACC</td></tr>
<tr><td>export NVCOMPILER_ACC_NOTIFY = 1|2|3</td><td>Environment variable for NOTIFY</td></tr>
</table>

…

5.1 Data Management in OpenACC

Below are examples of how different OpenACC data clauses control the movement and allocation of data between the CPU (host) and GPU (device):

1. Allocating memory on the GPU

```
int *A = (int*) malloc(N * sizeof(int));
#pragma acc parallel loop
{
    for (int i=0; i<N; i++) A[i] = 0;
}
```

In this snippet, memory for array `A` is allocated on the host using `malloc`. The OpenACC pragma without any explicit data clause does not allocate or manage the corresponding device memory. If `A` is not already present on the device, the compiler may implicitly manage this, but it's best to use data directives for clarity and control.

2. `copy(A[0:N])` – Copy data from host to device and back

```
for (int i=0; i<N; i++) A[i] = 0;
#pragma acc parallel loop copy(A[0:N])
{
    for (int i=0; i<N; i++) A[i] = 1;
}
```

The `copy(A[0:N])` clause tells the compiler to allocate space for array `A` on the GPU. Before the kernel runs, the data from the host (CPU) `A` is copied to the device (GPU). When the parallel region ends, the (possibly updated) data is copied back from device to host. This is useful when the GPU kernel is both reading from and writing to `A`, and you need to preserve the changes on the host after execution.

3. `copyin(A[0:N])` and `copyout(B[0:N])` – Directional data transfer

```
#pragma acc parallel loop copyin(A[0:N]) copyout(B[0:N])
{
    for (int i=0; i<N; i++) B[i] = A[i];
}
```

Here, `copyin(A[0:N])` means the data for `A` will be copied from the host to the device before the parallel region begins, but any modifications to `A` on the device are not copied back. `copyout(B[0:N])` means a fresh array `B` is allocated on the device, and after the region completes, the contents are copied from device to host. This is suitable when you need only to read from `A` on the device and produce output in `B`.

Data region constructs

…

Unstructured data

…

5.2 Loop Optimization

Seq clause

If we are in a situation where we have nested loops but we want to parallelize the outer loops, but not the inner one, we can use the `SEQ` clause.

```
#pragma acc parallel loop
for (int i = 0; i < N; i++) {
    #pragma acc loop
    for (int j = 0; j < M; j++) {
        #pragma acc loop seq
        for (int k = 0; k < K; k++) {
            A[i][j][k] = B[i][j][k] * C[i][j][k];
        }
    }
}
```

If we want to parallelize the outer loop, but not the inner one, we can use the `SEQ` clause.

Collapse clause

If you want to combine the next *N* tightly nested loops into a single loop, you can use the `collapse` clause. It is extremely useful for increasing memory locality, as well as creating larger loop to expose more parallelism.

```
#pragma acc parallel loop collapse(2)
for (int i = 0; i < N; i++) {
    for (int j = 0; j < M; j++) {
        // code
    }
}
```

Tile clause

If you want to parallelize a loop but you want to divide it into smaller chunks, you can use the `tile` clause. It is extremely useful for increasing memory locality, as well as executing multiple tiles simultaneously.

```
#pragma acc parallel loop tile(32,32)
for (int i = 0; i < 128; i++) {
    for (int j = 0; j < 128; j++) {
        // code
    }
}
```

Observation: ***Note***

Similar to the `collapse` clause, the inner loops should not have the `loop` directive.

Reduction clause

…

5.3 GPU Hardware Hierarchy

We have seen that the GPU is a very parallel machine, but it is not a single core machine. It's architechture is composed on Gangs, Workers and Vectors.

- **Gang**: Multiple gangs will be generated, and loops iterations will be spread across the gangs. Gangs are independent of each other. There is no way to for the programmer to know exactly how many gangs are running at a given time.
- **Worker**: To have multiple vectors within a gang. It splits up one large vector into multiple smaller vectors. It is an intermediate level between the low-level parallelism implemented in vector and group of threads. It is useful when our inner parallel loops are very small, and will not benefit from having a large vector.
- **Vector**: Lowest level of parallelism. Every gang will have at least 1 vector. Threads work in lockstep (SIMD/SIMT parallelism).

```
#pragma acc parallel loop gang
for (int i = 0; i < N; i++)
    #pragma acc loop worker
    for (int j = 0; j < M; j++)
        #pragma acc loop vector
        for (int k = 0; k < K; k++)
            // code
```

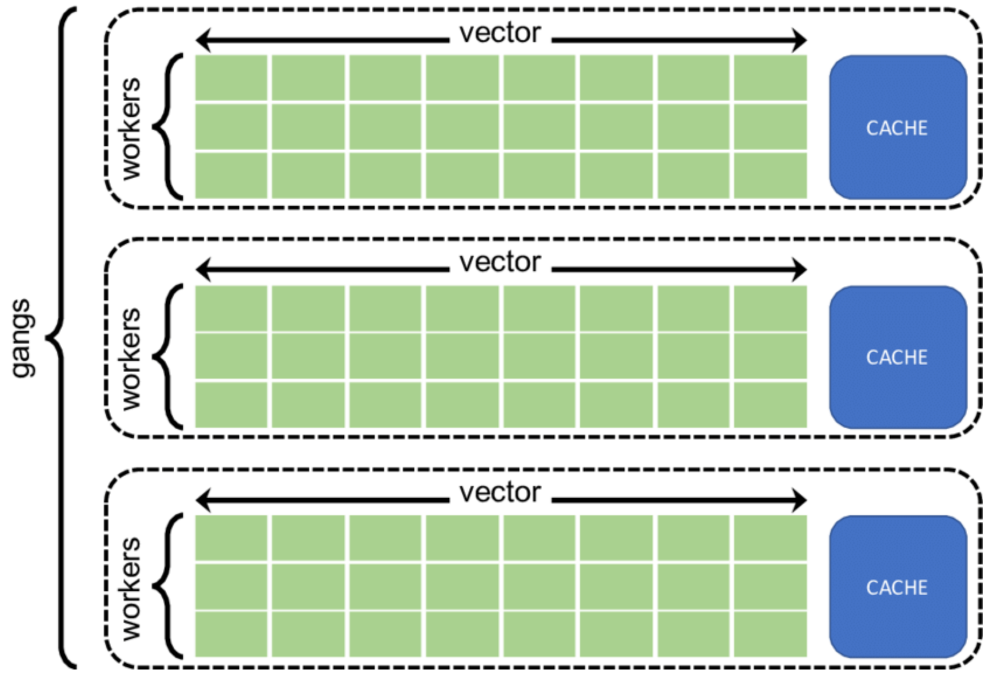


Figure 5.1: Gang, Worker and Vector Hierarchy

Tip: ***Rule of 32***

The rule of thumb to decide the gang, worker and vector sizes when programming for NVIDIA GPUs is to use a vector length that is a multiple of 32.

MISSING: good and bad example of usage

5.4 OMP vs ACC

OpenMP and OpenACC both provide high-level, directive-based approaches for parallel programming, allowing the developer to easily control how loops and computations are parallelized through the use of clauses. These models enable the compiler to map code onto multiple levels of parallelism, such as teams of threads or gangs, workers, and vectors, across different hardware architectures.

OpenACC	CUDA	Mapping to NVIDIA GPU	OpenMP
Parallel/Kernel	Kernel	GPU	Parallel
Gang	Thread block	SMs	Team
Worker	Thread	SP or Compute unit	Thread
Vector	Warp (32 threads)	32-wide thread	SIMD

Table 5.1: Comparison of Parallel Hierarchy: OpenACC, CUDA, NVIDIA GPU, and OpenMP

In theory (according to the standards) the implementation of the levels adapt to the hardware, but in reality some compilers struggle with certain parallelisation levels:

OpenACC	OpenMP
`acc parallel`	`omp target teams`
`acc loop gang`	`omp distribute`
`acc loop worker`	`omp parallel loop`
`acc loop vector`	`omp simd`
`acc declare`	`omp declare target`
`acc data`	`omp target data`
`acc update`	`omp target update`
`copy/copy_in/copy_out`	`map(to/from/tofrom: ... )`

Table 5.2: OpenACC to OpenMP Directive and Clause Correspondence

OMP Parallel

- Creates a team of threads
- Very well-defined how the number of threads is chosen
- May synchronize within the team
- Data races are the user's responsibility

...

ACC Parallel

- Creates 1 or more gangs to workers
- Compiler free to choose number of gangs, workers and vector lengths
- May not synchronize between gangs
- Data races are not allowed

6 MPI and GPU offloading using Python

6.1 MPI in Python

Message Passing Interface (MPI) is a standard for disrtribuited-memory parallel computing, using to enable communication between multiple processes. MPI is useful in HPC applications, where we need to distribuite large computations across multiple nodes.

MPI in python is implemented using the mpi4py library. To install it, we can use the following command:

```
pip install mpi4py
```

how to use:

```
mpirun -n <number_of_processes> python <script.py>
```

Communications

- **Point-to-point communications**

```python
if rank == 0:
    data = "Hellp, Process 1"
    comm.send(data, dest=1)
elif rank == 1:
    data = comm.recv(source=0)
    print(f"Process 1 received: {data}")
```

- **Non-blocking communications**

```python
if rank == 0:
    data = "Hellp, Process 1"
    comm.isend(data, dest=1, tag=11)
elif rank == 1:
    data = comm.irecv(source=0, tag=11)
    print(f"Process 1 received: {data}")
```

- **Collective communications**

```python
comm.bcast(data, root=0)
```

Example: *Example*

We want to implement a code that computes the distance between each point of a dataset and of another one, and then compute the distance distribution.

```python
import numpy as np
import mpi4py
from mpi4py import MPI

npart_data = int(sys.argv[1])
Nbins = 20

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

if rank == 0:
    Data1 = np.random.rand(npart_data, 2)
    Data2 = np.random.rand(npart_data, 2)
    bins = np.linspace(0, 1.5, Nbins)
else:
    Data1 = None
    Data2 = np.empty((npart_data, 2))
    bins = np.empty(Nbins)

comm.Barrier()

comm.Bcast(Data1, root=0)
comm.Bcast(Data2, root=0)

Data1_on_task = scatter_data(Data1)

dist_hist = get_statistical_distance([Data1_on_task, Data2, bins])

dist_hist_fin = none
if rank == 0:
    dist_hist_fin = np.empty([size, len(dist_hist)])

comm.Gatherv(dist_hist, dist_hist_fin, root=0)

if rank == 0:
    dist_hist_fin = np.sum(dist_hist_fin, axis=0)
```

where `scatter_data` is a function that scatters the data to the different tasks.

```python
def scatter_data(Data):
    rows_per_rank = np.full(size, npart_data // size)
    rows_per_rank[:npart_data % size] += 1

    # Allocate memory for the data on the current task
    Data1_on_task = np.empty((rows_per_rank[rank], 2))

    # Calculate the displacement
    displacement = np.cumsum(np.insert(rows_per_rank[:-1], 0, 0)) *
        2

    # Scatter the data
    comm.Scatterv([Data1, rows_per_rank * 2, displacement, MPI.
       DOUBLE], Data1_on_task, root=0)

    return Data1_on_task
```

6.2 GPU offloading in Python

It is possible to offload the computation to the GPU using different libraries.

6.2.1 Low level APIs: PyCUDA, PyOpenCL

PyCUDA is a library that allows to write CUDA code in Python. It provides a low level API for CUDA programming.


```
...
```

6.2.2 Mid level APIs: Numba

Numba is a just-in-time compiler for Python that allows to write CUDA code in Python. It provides a mid level API for CUDA programming.

```
...
```

6.2.3 High level APIs: CuPy

CuPy is an open-source array library for GPU-accelerated computing in Python on NVIDIA CUDA or AMD ROCm platforms.

It is highly comparible with NumPy and SciPy; in most cases it can be used as a drop-in replacement.

At the same time it allows programmers to write custom CUDA kernels.

```python
import cupy as cp

def distances_GPU_cupy(Data1, Data2, bins):
    Data1 = cp.asarray(Data1)
    Data2 = cp.asarray(Data2)
    bins = cp.asarray(bins)

    dist_hist = get_distances_GPU_cupy(Data1, Data2, bins)
```

```python
    return dist_hist

def get_distances_GPU_cupy(Data1, Data2, bins):
    # calculate the distances in a vectorized manner
    dx = Data1[:, 0][:, cp.newaxis] - Data2[:, 0][cp.newaxis, :]
    dy = Data1[:, 1][:, cp.newaxis] - Data2[:, 1][cp.newaxis, :]
    dist = cp.sqrt(dx**2 + dy**2)

    # use cp.histogram for GPU histogramming
    dist_hist = cp.histogram(dist, bins=bins)
    return dist_hist
```

Extensions

- cuDF is a GPU DataFrame library for managing data, that extends the capabilities of Pandas to the GPU.
- Dpnp is a library used by Intel that implements a subset of the NumPy API that can be executes on any data parallel device.

> **Warning**: ***Offloading only when needed***
>
> Offloading only when needed. If the data is small, it is not worth offloading it to the GPU, and it can take more time than the computation on the CPU.

7
Message Passing Interface - MPI

Advanced Usage of some MPI features on Topology awareness, Shared memory and One-sided communications.

- Basics of the One-sided communication
- Buildinga hierarchy of Communicators that reflects the topology
- Exchanging data in shared memory windows among MPI processes

and we will see briefly other advanced features of MPI:

- Derived Data-Types
- More details on Groups and Communicators
- Virtual topologies (cartesian and graph communicators) and neighborhood collectives
- Non-blocking collectives

Two-Sided communications

By its very nature, the message-passing paradigm is designed around the concept of cooperative exchange of informations among two or more processes whose address space are isolated and not directly inaccessible by other processes.

7.1 Remote Memory Access - RMA

7.1.1 Memory Windows

Warning: *`disp_unit`*

The `disp_unit` parameter is often misunderstood. Setting it to anything other than 1 (byte) can cause subtle bugs when mixing datatypes. The displacement calculation becomes:

```
target_addr = base + (target_disp * disp_unit)
```

Observation: *Flush vs Sync*

The fundamental difference between `MPI_Win_flush` and `MPI_Win_sync` is that:

- `MPI_Win_flush`: Forces completion of RMA operations from the origin to the target.
- `MPI_Win_sync`: Synchronizes local memory with the window (target-side operation).

Or, in other words:

- `MPI_Win_flush` is about *Network Completion* (*Did the data arrive?*)
- `MPI_Win_sync` is about *Memory Consistency* (*Is the data visible to the CPU?*)

This distinction is critical and often misunderstood: these calls operate on different sides of the communication and serve completely different purposes.

Locks

Locks are a way to synchronize the access to a shared resource. They are a way to ensure that only one process can access the shared resource at a time.

Passive mode: the target does not participate in opertions: one sided asynchronous communication.

```
MPI_Win_lock(int lock_type, int rank, int assert, MPI_Win win);

MPI_Win_unlock(int rank, MPI_Win win);

MPI_Win_flush/flush_local(int rank, MPI_Win win);
```

where `lock_type` is one of the following:

- `MPI_LOCK_EXCLUSIVE` : only one process can access the shared resource at a time.
- `MPI_LOCK_SHARED` : multiple processes can access the shared resource at a time.

`rank` is the rank of the process that will access the shared resource.

`assert` is the assertion that will be used to synchronize the access to the shared resource.

- `MPI_Win_flush` : complete operations on remote target process; data will be then available to the target task, or to other tasks
- `MPI_Win_flush_local` : locally complete operations to the target process

⚠ Warning: ***Misconception about locks***

Despite the unfortunate naming, MPI locks are not mutex-like. They do not provide mutual exclusion for the target process's local access. The target can modify its window memory during a locked epoch, leading to race conditions.

Assertions

7.2 Shared Memory

An HPC machine is made of several nodes that are connected by a top-level network We call these nodes "hosts".

Currently every host contains a NUMA region which consists in a number of sockets. Each of which is physically connected to RAM banks. In turn, the sockets are interconnected so that a process running on a given core can access a memory bank not physically attached to its socket.

7.2.1 Getting Shared Memory Pools

General and specific routines

MPI provides a method to know about that and to distinguish the tasks depending some of their charateristics:

- a `general routine` to split a communicator into sub-communicators:

```
MPI_Comm_split (
    MPI_Comm comm,      // the communicator to be splitted
    int color,          // discriminate task
    int key,            // hint for new ranks
```

```c
    MPI_Comm *newcomm   // the new communicator
);
```

- a `specific routine` to split a communicator into sub-communicators following the hardware hierarchy (i.e. NUMA region, socket, core, etc.):

```c
MPI_Comm_split_type (
    MPI_Comm comm,      // the communicator to be splitted
    int split_type,     // the splitting property
    int key,            // hint for new ranks
    MPI_Info info,      // helper for the splitting property
    MPI_Comm *newcomm   // the new communicator
);
```

  The `split_type` parameter is used to specify the splitting property. It can be one of the following values:

  – `MPI_COMM_TYPE_SHARED` : split the communicator based on the shared memory pools.

  – `MPI_COMM_TYPE_HW_GUIDED` : split the communicator based on the hardware guided property.

  – `MPI_COMM_TYPE_HW_UNGUIDED` : split the communicator based on the hardware unguided property.

  The `key` parameter is used to hint for the new ranks. It is used to sort the tasks depending on their charateristics.

  The `info` parameter is used to provide helper for the splitting property. It is used to provide additional information about the splitting property.

Placement

```c
MPI_Win_allocate_shared (
    MPI_Aint size,          // the size of the window
    int disp_unit,          // the displacement unit
    MPI_Info info,          // the information to be used
    MPI_Comm comm,          // the communicator
    void *baseptr,          // the base pointer
    MPI_Win *win            // the window
);
```

As with other window-creation routines, the memory allocation for `MPI_Win_allocate_shared` does not require the allocated size to be uniform: each task may allocate a different amount. There is no strict specification as to where the memory should be physically placed: an MPI implementation will typically attempt to maximize data locality and affinity.

By default, the allocation is **contiguous**. However, this contiguity does not only refer to the virtual address space: ideally, it also implies allocation in the same physical memory bank. On many systems, though, it is not possible to remap memory at arbitrary sizes to guarantee this property. Specifying a non-contiguous allocation can allow the MPI implementation to more flexibly optimize the placement, possibly enhancing the affinity for each individual task.

A useful routine related to window allocation and memory sharing is `MPI_Win_shared_query` , which allows processes to retrieve a pointer to shared memory in a window, for either direct access or load/store operations. Its prototype is:

```c
MPI_Win_shared_query(
    MPI_Win   win,          // the window object
    int       rank,         // rank whose segment to query, or MPI_PROC_NULL
    MPI_Aint *size,         // returns segment size
    int      *disp_unit,    // returns displacement unit
    void    **baseptr       // returns base pointer in shared memory
);
```

This function enables a process to obtain the base address and size of the memory segment exposed by any rank in a shared-memory communicator window. Commonly, it is called with `rank = MPI_PROC_NULL`, which gives a pointer to the start of the entire shared memory region (regardless of which process allocated it), and returns:

- the address of the first byte in the shared memory window (for contiguous allocation), or
- the base address of the first process that specified a non-zero `size` (for non-contiguous allocation).

The figure above illustrates the difference between default contiguous allocation and non-contiguous allocation across multiple processes. With contiguous allocation, addresses increase sequentially across processes, so the pointer returned can be used by all processes to access the underlying shared array as a unified space. In non-contiguous mode, each process's memory may be mapped separately, and thus, only the base address of the first valid allocation is returned.

This routine is fundamental for writing portable and generic code utilizing shared memory windows, especially when contiguity cannot be assumed.

7.3 Cartesian Communication

Many scientific and engineering problems operate on regular, structured grids: finite difference methods for PDEs, stencil computations, image processing, and particle-mesh methods in astrophysics. When parallelizing these applications via domain decomposition, we partition the computational domain into sub-domains, each handled by a distinct MPI process.

The fundamental challenge is neighbor communication: processes must exchange boundary data (ghost zones, halo cells) with their neighbors. While we could manually compute neighbor ranks using arithmetic on process coordinates, this approach is error-prone, non-portable, and fails to leverage potential hardware optimizations.

Why not Manual Rank Arithmetic

Consder a 2D grid decomposition with dimensions $(P_x \times P_y)$. Given a process with coordinates (i, j), its neighbors are:

```c
// Manual neighbor calculation (fragile approach)
int rank_left  = (i > 0)     ? i-1 + j*Px   : MPI_PROC_NULL;
int rank_right = (i < Px-1)  ? i+1 + j*Px   : MPI_PROC_NULL;
int rank_down  = (j > 0)     ? i + (j-1)*Px : MPI_PROC_NULL;
int rank_up    = (j < Py-1)  ? i + (j+1)*Px : MPI_PROC_NULL;
```

The possible issues, or inconveniencies, with this approach are:

- **Boundary handling**: Explicit conditionals for edges; periodic boundaries add complexity
- **Dimension ordering**: Row-major vs column-major rank assignment affects all calculations

- **Scalability**: 3D or higher dimensions require nested conditionals
- **Optimization**: MPI implementations cannot optimize message routing without knowing the topology

Cartesian Communication Model

MPI provides virtual topologies to express logical process arrangements. A **Cartesian topology** maps processes onto a regular n-dimensional grid, and MPI provides a suite of routines for creating and querying such topologies:

- **Automatic coordinate ↔ rank translation**: Convert between process ranks and their multidimensional coordinates in the grid.
- **Built-in periodic boundary support**: Specify whether each grid dimension is periodic (wraparound), e.g., for toroidal grids.
- **Neighbor lookup via relative displacement**: Functions for obtaining neighbor ranks given directional offsets ("shifts")—no manual arithmetic required.
- **Hints for process placement optimization**: Inform the MPI implementation of communication patterns, potentially improving performance by optimizing process placement (implementation-dependent).

Example: *Non-Periodic 3x2 Grid*

```
          j=0    j=1
        +------+------+
  i=0   |  R0  |  R1  |   Rank = i + j*3
        +------+------+
  i=1   |  R2  |  R3  |   R0:(0,0) R1:(1,0)
        +------+------+   R2:(0,1) R3:(1,1)
  i=2   |  R4  |  R5  |   R4:(0,2) R5:(1,2)
        +------+------+
```

7.3.1 Core API Functions

MPI_Dims_create

Computes a "balanced" distribution of processes across dimensions:

```c
int MPI_Dims_create (int nnodes,    // Total number of processes
                     int ndims,     // Number of dimensions
                     int dims[]);   // IN/OUT: dimension sizes
```

MPI_Cart_create

Creates a new communicator with processes laid out on a Cartesian grid:

```c
int MPI_Cart_create(MPI_Comm comm_old, int ndims, const int dims[],
    const int periods[], int reorder, MPI_Comm *comm_cart);
```

Key behavior: Pre-set dimensions (dims[i] > 0) are preserved; zero entries are computed.

The algorithm tries to minimize surface-to-volume ratio for communication efficiency.

Example: *2D grid, 12 processes*

```c
// unconstrained: let MPI choose
int dims[2] = {0, 0};
MPI_Dims_create(12, 2, dims); // Result: dims = {4, 3} or {3, 4}

// constrained: force 4 columns
int dims2[2] = {0, 4};
MPI_Dims_create(12, 2, dims2); // Result: dims2 = {3, 4}
```

Warning: *MPI_Dims_create failure*

MPI_Dims_create fails if `nnodes` is not divisible by the product of pre-set dimensions. Always verify `dims[i] > 0` for all `i` after the call.

MPI_Cart_create

Creates a new communicator with Cartesian topology:

```c
int MPI_Cart_create(
    MPI_Comm comm_old,       // Input communicator
    int ndims,               // Number of dimensions
    const int dims[],        // Size of each dimension
    const int periods[],     // Periodicity flags (0 or 1)
    int reorder,             // Allow rank reordering?
    MPI_Comm *comm_cart      // Output: new Cartesian comm
);
```

where:

- `periods[i]` : If non-zero, dimension `i` wraps around (periodic boundary)
- `reorder` : If non-zero, MPI may reassign ranks to optimize for hardware topology
- If size(`comm_old`) > product(`dims`), excess processes receive `MPI_COMM_NULL`

Tip: *Best practices*

Set reorder=1 in production code. This allows MPI implementations (especially on clusters with complex interconnects like fat-trees or torus networks) to map logically adjacent processes to physically adjacent nodes, reducing communication latency.